



Decision Making



Decision Making

È un processo centrale nello sviluppo di AI nei videogiochi.

Si riferisce alla capacità di un'entità AI di scegliere l'azione migliore da intraprendere in una determinata situazione, sulla base di regole, dati raccolti o comportamenti desiderati.

Questo processo è fondamentale per rendere i personaggi non giocatori (NPC) reattivi e credibili, migliorando l'immersione del giocatore.



Decision Making

In generale, possiamo immaginare il personaggio controllato dall'AI come ad un'entità che **reagisce** a:

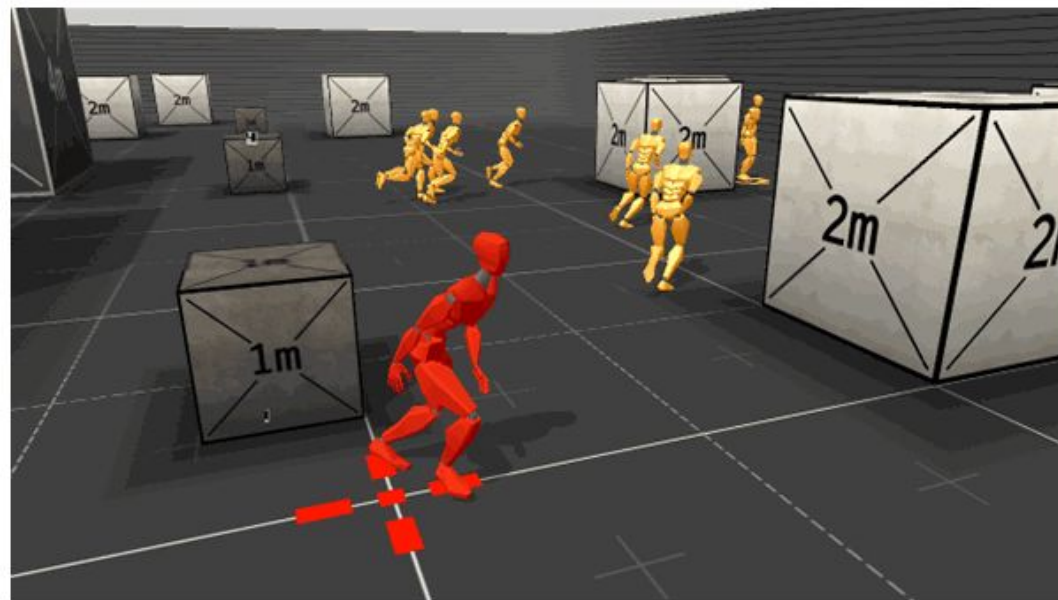
- **Stimoli interni** (*stato*)
- **Stimoli esterni** (*input*)

Questi stimoli possono essere rappresentati da diverse variabili (livello della vita, distanza dei nemici, munizioni, etc.)

La AI reagirà a delle **condizioni** su queste variabili come:

- Ho abbastanza vita?
- Vedo dei nemici?
- Sto subendo danni?
- L'arma è carica?

Valutate le condizioni, l'AI effettuerà delle **azioni**.

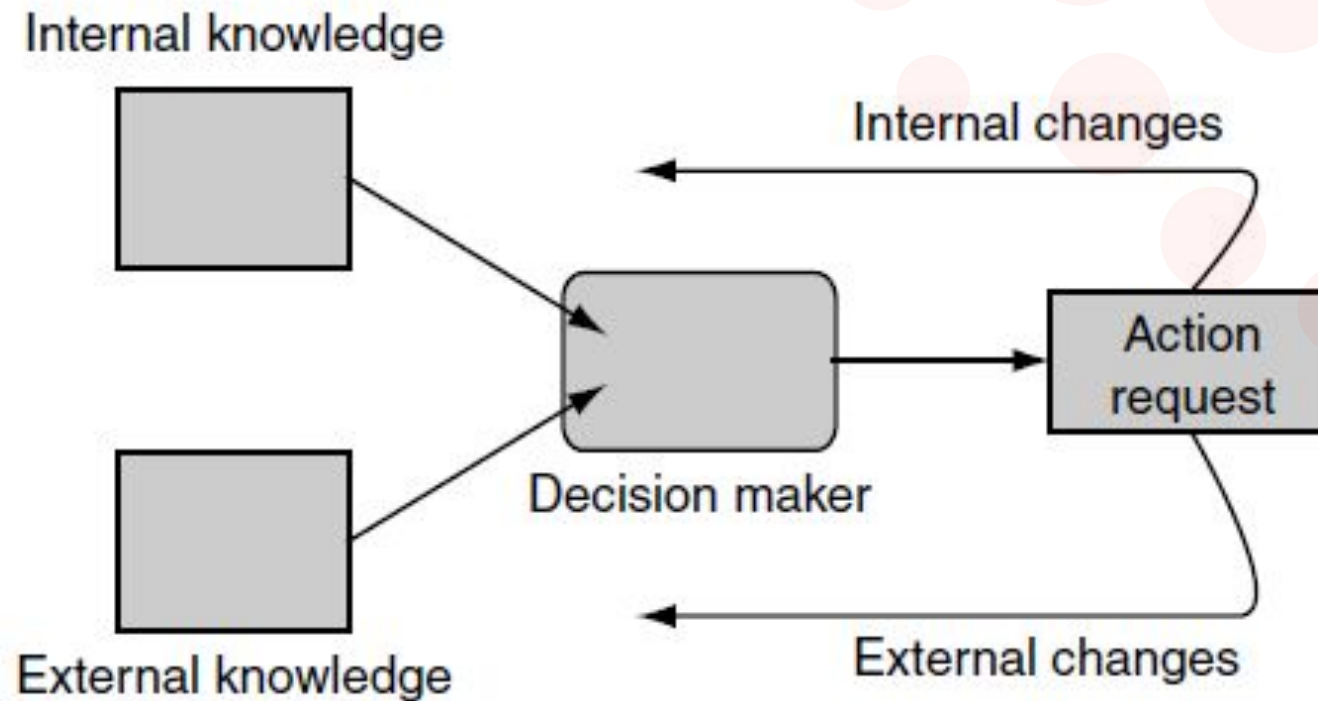




Decision Making

Possiamo quindi riassumere i componenti del decision making in 4 fasi:

1. Raccolta di informazioni
2. Elaborazione delle informazioni
3. Selezione delle azioni
4. Esecuzione delle azioni





Decision Making - Obiettivi

Quali sono quindi gli **obiettivi** che si vuole raggiungere?

- Creare NPC controllati da un AI che reagisca agli stimoli in modo **verosimile**
- Gli NPC devono reagire ai diversi stimoli in modo ben definito dai designer, avendo quindi un AI facilmente **configurabile**
- Vogliamo che l'AI reagisca in modo **deterministico** secondo una logica data

Spesso, non abbiamo bisogno di un'intelligenza troppo avanzata, in quanto solitamente il giocatore non riesce a distinguere un'intelligenza semplice da una complessa.

Al contrario, tecniche complesse possono essere difficili da rendere configurabili e deterministiche oltre che poco performanti.

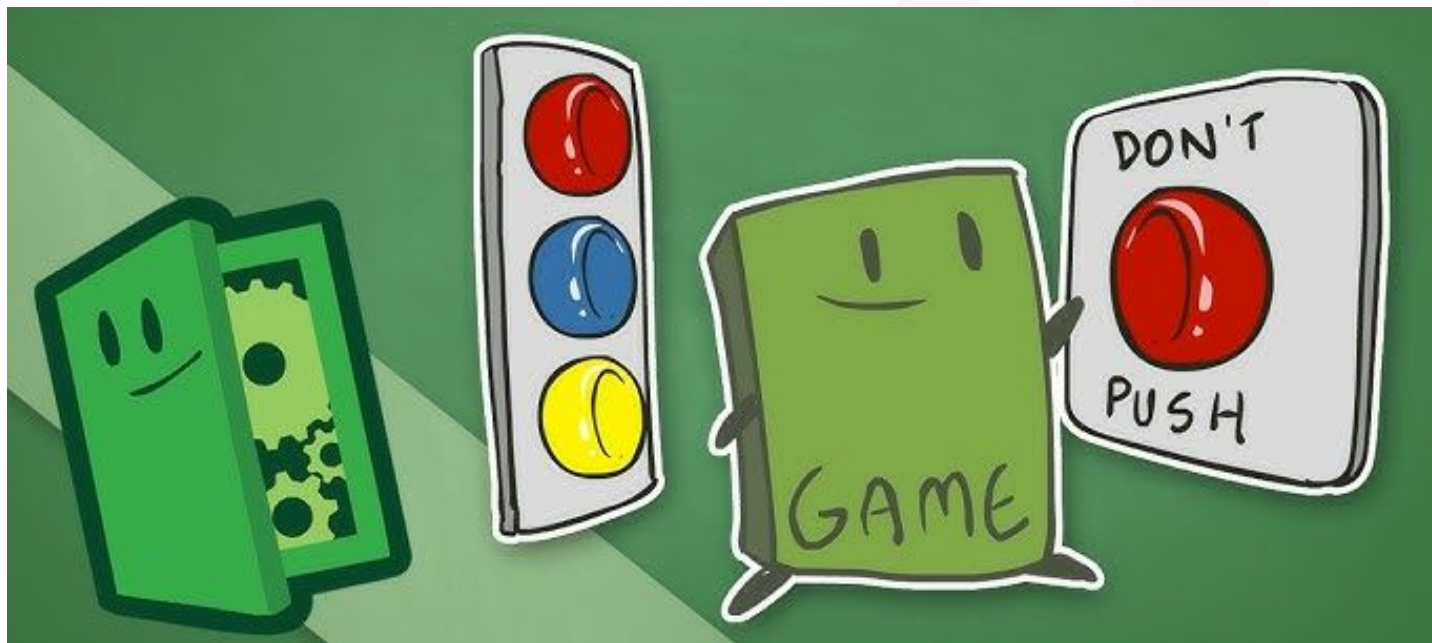
In fondo, il più delle volte l'importante è che il giocatore riesca o comprenda almeno come poter rispondere al pattern di reazione dei nemici.





Decision Making - Tecniche

Esistono diverse soluzioni e tecniche di decision making che permettono all'AI di scegliere l'azione più appropriata in base al gioco o alla situazione in cui si ritrovano. Esse possono essere più o meno complesse.



Un esempio di soluzione semplice, specifiche per alcuni giochi o situazioni di gioco sono le cosiddette **situazioni ad hoc**.

In questo caso il confine tra AI e Gameplay Programming è molto labile.



Cutscenes/Sequences

Un esempio di soluzione ad hoc sono le **Cutscenes o Sequences**, ovvero scene in cui i personaggi eseguono una serie predefinita e statica di azioni in sequenza, che possono derivare dal flusso di gioco o da una scelta del giocatore, ma non lascia agli NPC alcuna capacità effettiva di reazione

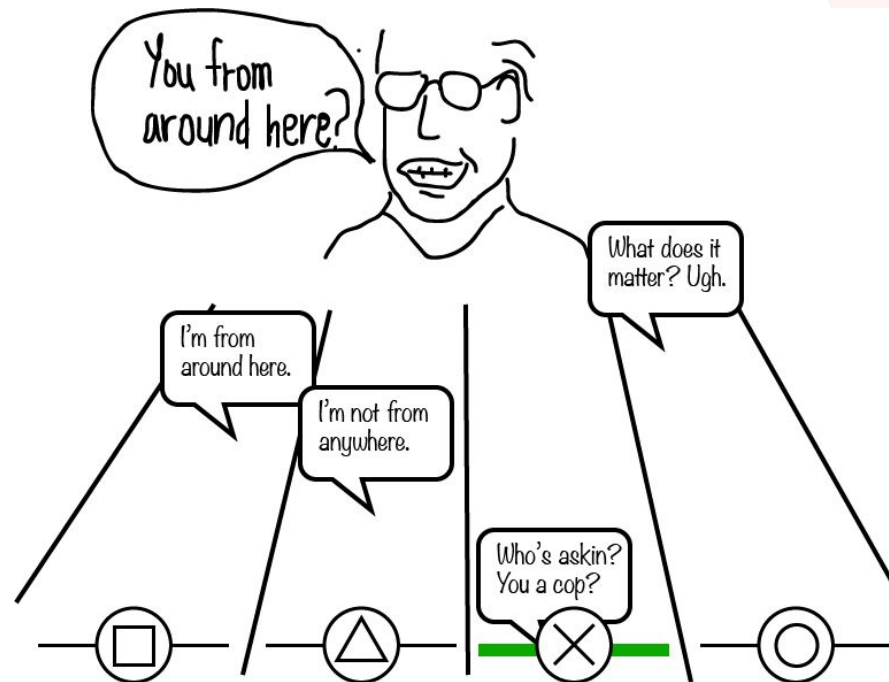




Dialogue Tree

Un altro esempio di soluzione ad hoc sono i **Dialogue Tree**, una struttura narrativa utilizzata per gestire le conversazioni interattive tra il giocatore e gli NPC. Essi rappresentano una serie di opzioni di dialogo che il giocatore può scegliere, con ciascuna scelta che porta a risposte o azioni diverse da parte dell'NPC, creando una conversazione ramificata.

In questo caso il personaggio ha una minima capacità di reazione, ma è totalmente illusoria.



RPGs then...



RPGs now...





Patterns

Un ultimo esempio sono i **patterns**, sequenze predefinite di azioni/reazioni che un'AI può adottare in base a situazioni specifiche, innescato in base a tempistiche o posizione del giocatore ad esempio.

In questo caso, il personaggio ha una minima capacità di reazione, spesso prevedibile e/o strategico.





Ad Hoc - Svantaggi

Alcuni svantaggi che tutti questi esempi di soluzioni ad hoc portano sono abbastanza palesi, come ad esempio la non riutilizzabilità, in quanto ogni sistema è implementato per uno specifico gioco.

Allo stesso momento non esistendo delle teorie o tecniche standard, ogni eventuale problema di sviluppo è lasciato alla specifica implementazione, il che porta a dover utilizzarle per giochi molto semplici, altrimenti alcune situazioni rischiano di esplodere in complessità.





Rule-based System

Un altro approccio utilizzabile, che si differisce dalle soluzioni ad hoc, andando a creare possibili multiple situazioni e reazioni da parte degli NPC è un sistema *IF-ELSE* semplice, detto a **regole condizionali** (rule-based).

La base per creare una condizione logica è proprio l'uso di *IF-ELSE*:

```
if (enemyIsClose)
{
    if (weHaveSuperPower) UseSuperPower();
    else if (weHaveWeapon) Attack();
    else Flee();
}
```



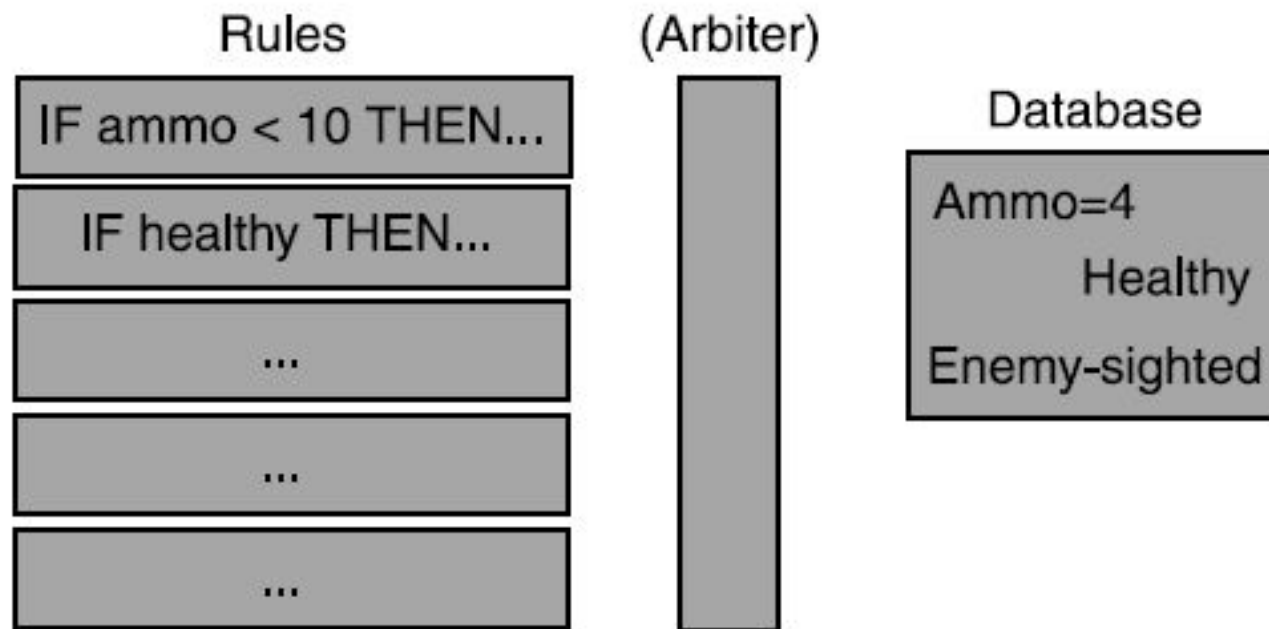

Rule-based System

Estrapolando da una serie di *IF-ELSE*, definiamo un sistema rule based con:

- Una serie di regole (eventualmente scriptate)
- Una database di variabili

Basterà eseguire le regole una dopo l'altra!

Chiamato anche **Expert System**.





Rule-based System - Esempio



```
17075 (set-strategic-number sn-food-gatherer-percentage 34)
17076 (set-strategic-number sn-wood-gatherer-percentage 52)
17077 (set-strategic-number sn-gold-gatherer-percentage 14)
17078 )
17079
17080 (defrule
17081 (up-compare-goal gl-current-age >= gv-feudal-up)
17082 (current-age == castle-age)
17083 (current-age-time >= 240)
17084 (current-age-time < 600)
17085 (building-type-count-total town-center > 2)
17086 (or (strategic-number sn-food-gatherer-percentage != 41)
17087     (or (strategic-number sn-wood-gatherer-percentage != 45)
17088         (strategic-number sn-gold-gatherer-percentage != 14))))
17089 (goal POSITION POCKET)
17090 =>
17091 (set-strategic-number sn-food-gatherer-percentage 41)
17092 (set-strategic-number sn-wood-gatherer-percentage 45)
17093 (set-strategic-number sn-gold-gatherer-percentage 14)
17094 )
17095
17096 (defrule
17097 (goal gl-strategy xbow-boom)
17098 (up-compare-goal gl-current-age >= gv-feudal-up)
17099 (current-age >= castle-age)
17100 (or (current-age-time > 600)
17101     (or (up-compare-goal gl-current-age >= gv-castle-up)
17102         (or (up-compare-goal DOCK != 0)
```

<https://gist.github.com/Andygmb/1e3a6d9d444b2dfa8c40>



Rule-based System - Svantaggi

Anche se la logica di azione/reazione dei personaggi, tramite il sistema rule-based, può essere totalmente “*scriptata*” da un designer usando un linguaggio di scripting testuale o visuale, questo porterà alla necessità di richiedere ad un designer di imparare a programmare!

Inoltre, nonostante la possibilità che questo sistema abbia una grande capacità di configurazione e controllo, esso risulta poi molto verboso, di conseguenza error-prone e poco mantenibile.

Da segnalare anche la difficoltà nel ragionare al volo sul comportamento delle AI, in quanto bisogna andare a considerare tutte le combinazioni per cercare di ottenere un dato risultato, mantenendo un’alta difficoltà nel definire una relazione tra le diverse regole.

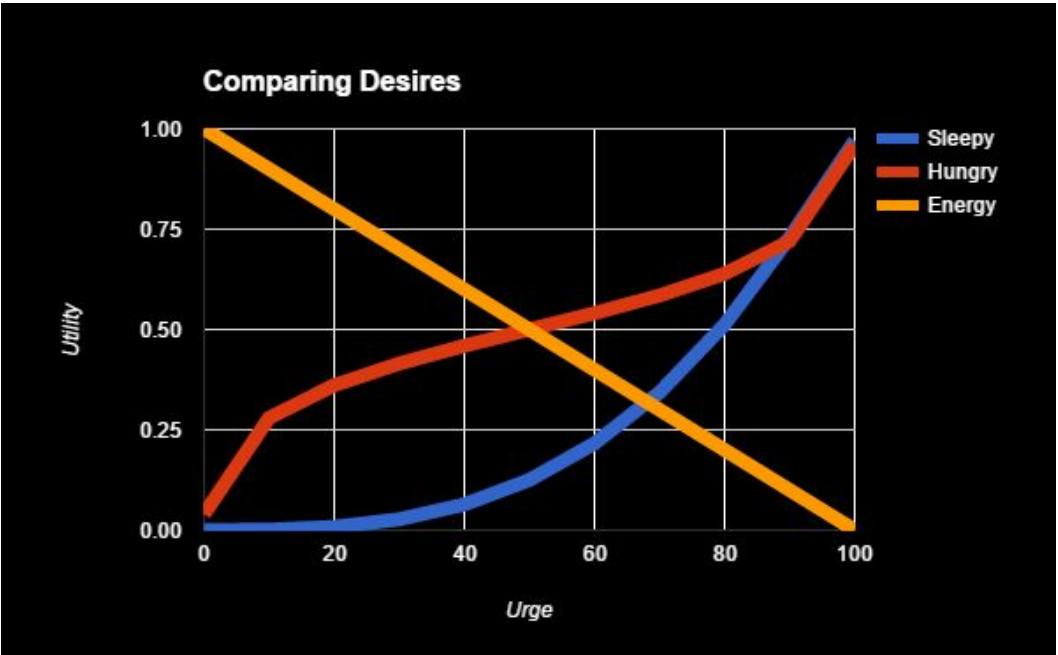


Utility AI

L'Utility AI è un approccio all'intelligenza artificiale nei videogiochi che si basa sul calcolo di valori numerici chiamati "utility values" per determinare quale azione un NPC debba eseguire in un dato momento.

Questo approccio consente un comportamento flessibile e adattabile, ideale per situazioni in cui ci sono molte possibili azioni tra cui scegliere e vari fattori da considerare.

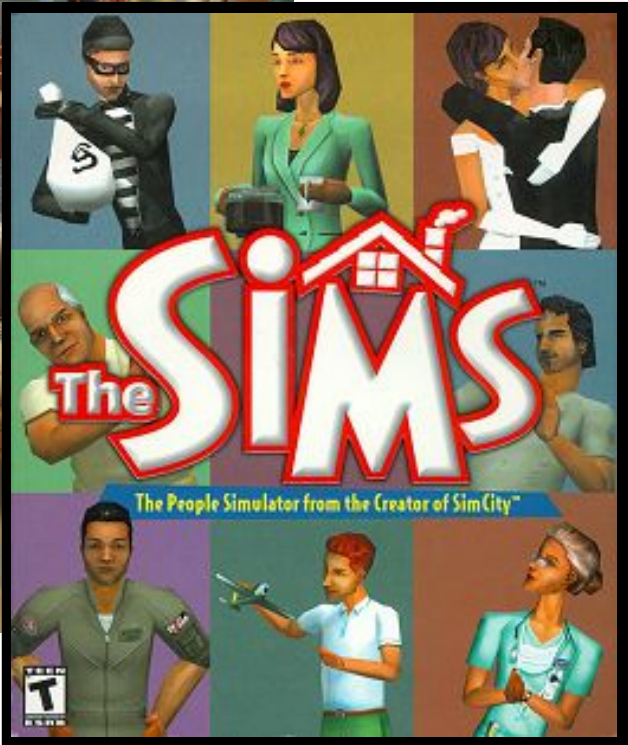
L'Utility AI valuta le possibili azioni in base a una serie di criteri (ad esempio, bisogni, obiettivi o contesto) e assegna a ciascuna azione un punteggio di utilità. L'azione con il punteggio più alto viene scelta ed eseguita.



Object	Interaction	Score
Fridge	Make Lunch	65
Computer	Play Game	45
TV	Watch	35
Chair	Sit	8
Sink	Clean	3
Bed	Sleep	3
Couch	Nap	2



Utility AI - Esempio

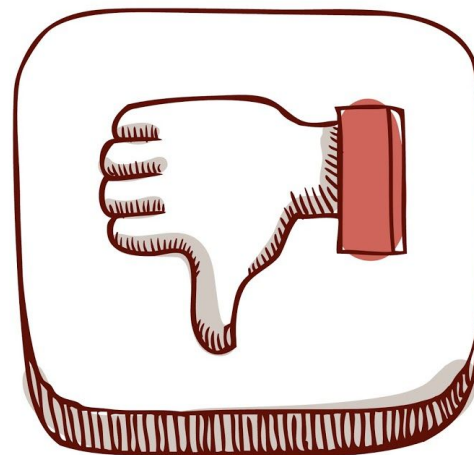




Utility AI

Vantaggi:

- Realismo: Gli NPC appaiono più intelligenti e credibili, adattandosi a diverse situazioni.
- Adattabilità: Funziona bene in contesti con molteplici fattori e azioni possibili.
- Facilità di espansione: È semplice aggiungere nuovi criteri o regolare i valori esistenti.
- Comportamento fluido: Permette transizioni naturali tra le azioni senza stati rigidi.



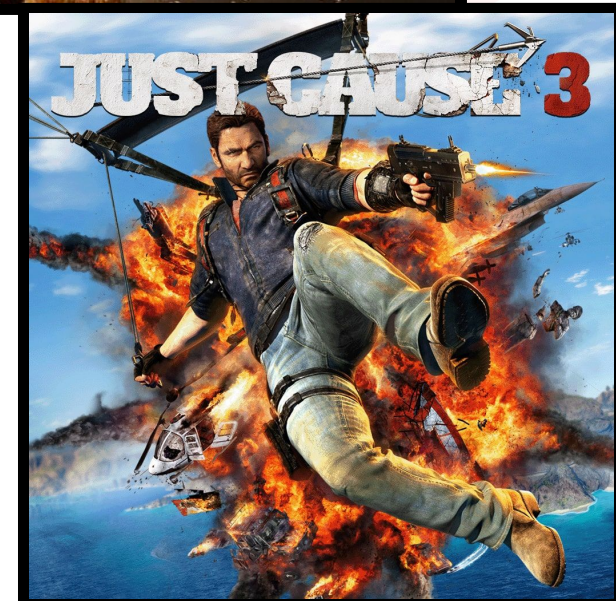
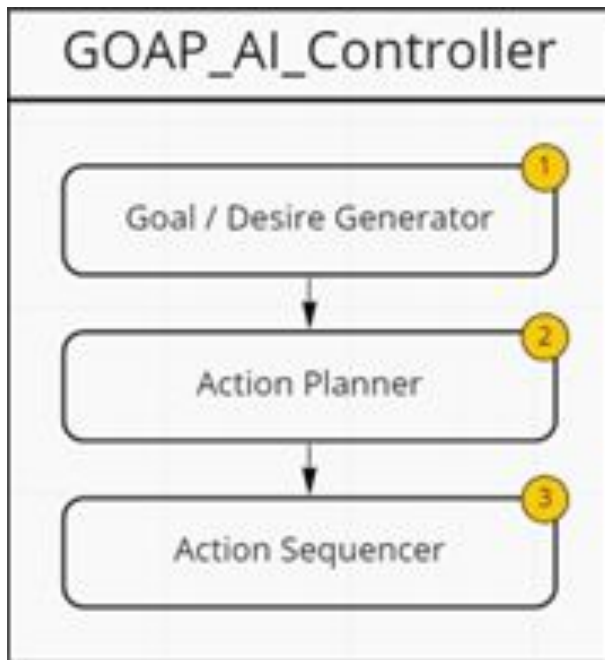
Svantaggi:

- Complessità matematica: Richiede una buona conoscenza di funzioni per bilanciare i valori.
- Costi computazionali: Valutare molte azioni e criteri può essere oneroso in termini di prestazioni.
- Manutenzione: Può diventare difficile gestire molte regole e parametri in giochi complessi.



GOAP

Estensione di Utility AI, il paradigma **Goal-Oriented Action Planning**, detto *GOAP*, permette agli NPC di prendere decisioni basate su obiettivi e di pianificare le azioni necessarie per raggiungerli.

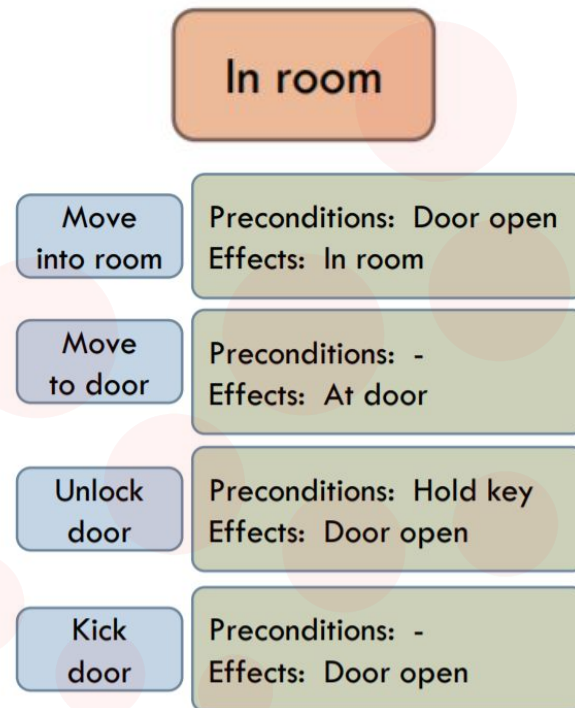
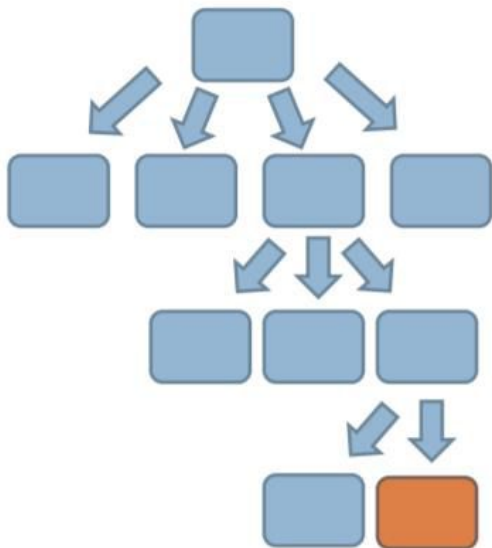




GOAP

Come funziona:

- **Definizione degli obiettivi**: Gli NPC hanno una lista di obiettivi (ad esempio, "Sopravvivere", "Attaccare il nemico", "Entrare in stanza").
- **Azioni disponibili**: Ogni azione parte dallo stato del gioco e si definisce tramite delle **precondizioni** (ciò che deve essere vero per eseguire l'azione) per poi scaturirne degli **effetti** (ciò che sarà vero dopo che l'azione è stata completata).



- **Pianificazione**: L'NPC utilizza degli algoritmi per creare un piano che collega azioni successive, soddisfacendo tutte le regole e portando al raggiungimento dell'obiettivo.
- **Esecuzione**: L'NPC esegue il piano, adattandosi dinamicamente a eventuali cambiamenti nel mondo di gioco.



GOAP

Vantaggi:

- Flessibilità: Gli NPC possono reagire a scenari complessi e dinamici, adattando il piano se l'ambiente cambia.
- Realismo: Gli NPC appaiono più intelligenti e motivati, poiché le loro azioni sono guidate da obiettivi chiari.
- Riutilizzabilità: Una singola struttura GOAP può essere utilizzata per diversi tipi di NPC, riducendo i costi di sviluppo.



Svantaggi:

- Costi computazionali: Pianificare azioni in tempo reale può essere intensivo, specialmente in giochi con molti NPC.
- Implementazione complessa: Richiede un'architettura solida e ben progettata per evitare bug o comportamenti imprevedibili.
- Richiede tuning: La definizione delle precondizioni, postcondizioni e costi delle azioni deve essere accuratamente bilanciata.





GOAP - Esempio

Usiamo GOAP per creare l'AI di un taglialegna.

Azioni:

- Può raccogliere legni
- Può tagliare alberi, se ha un'ascia, ricavando 4 legni
- Tramite 4 legni, Può creare un'ascia, se non ce l'ha già

Stato:

- Wood - Totale di legno raccolto
- Durability - Resistenza dell'ascia (valore di default 5)
 - Se =0, l'ascia è rotta e bisogna crearne un'altra
 - Se >0, puoi usare l'ascia per tagliare un albero (toglie 1 punto)

Obiettivo:

- Raccogliere 30 risorse di legno





GOAP - Esempio

Azione	Descrizione



GOAP - Esempio

Preconditions	Wood	Durability
CollectWood		
CraftAxe		
ChopTree		



GOAP - Esempio

Effects	Wood	Durability
CollectWood		
CraftAxe		
ChopTree		

Vediamo come potremmo implementare il codice di queste Azioni partendo da una classe base...



GOAP - Esempio

```
public abstract class Action
{
    public void Init
    {
        base.Init();
        preconditions.Empty();
        effects.Empty();
    }

    public void Perform(Agent agent)
    {
        agent = NULL;
    }
}
```



GOAP - Esempio

```
public class Action_CollectWood : Action
{
    public override void Init
    {
        base.Init();
        effects["wood"] = 1;
    }

    public override void Perform(Agent agent)
    {
        agent.currentWood += 1;
    }
}
```




GOAP - Esempio

```
public class Action_CraftAxe : Action
{
    public override void Init
    {
        base.Init();
        preconditions["durability"] = 0;
        preconditions["wood"] = 4;
        effects["wood"] = -4;
        effects["durability"] = 5;
    }

    public override void Perform(Agent agent)
    {
        agent.currentWood -= 4;
        agent.AddAxe();
    }
}
```

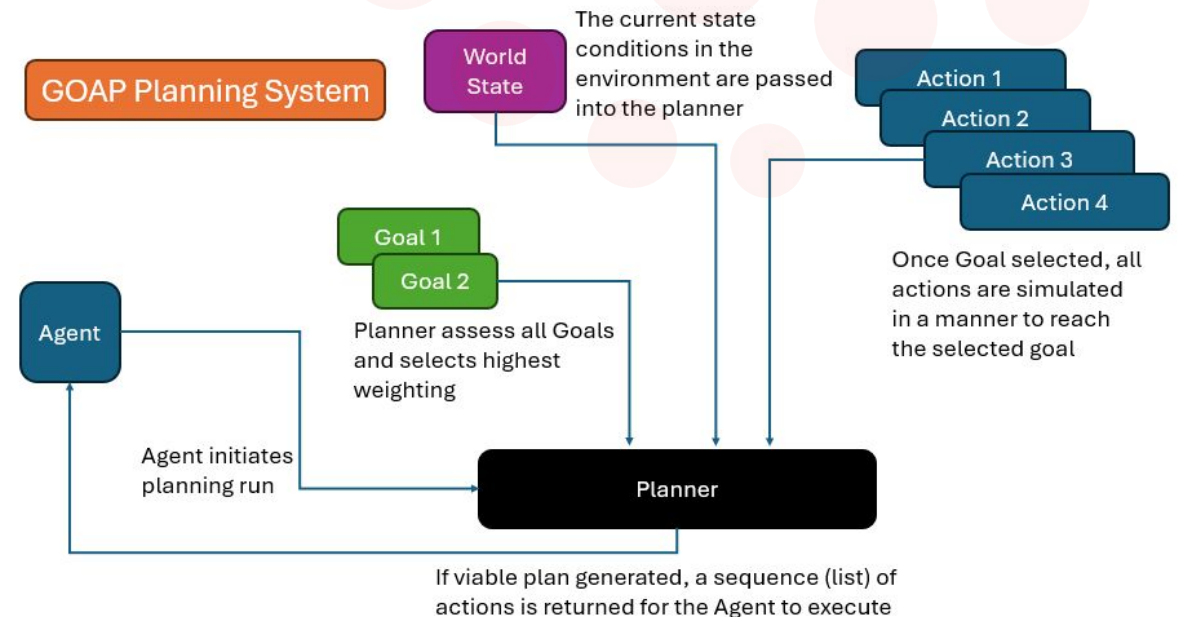


GOAP

Nella pianificazione abbiamo parlato di utilizzo di algoritmi.

Per una ricerca verso lo stato finale, un **algoritmo base** può prevedere i seguenti step:

- Partiamo dallo stato corrente
- Aggiungiamo lo stato alla coda
- Per ogni stato nella coda:
 - Recuperiamo ogni azione possibile, date le precondizioni
 - Generiamo gli stati successivi, applicando gli effetti delle azioni
 - Otteniamo gli stati successivi e li aggiungiamo alla coda
 - Se abbiamo raggiunto lo stato finale, terminiamo
 - Altrimenti, ripetiamo



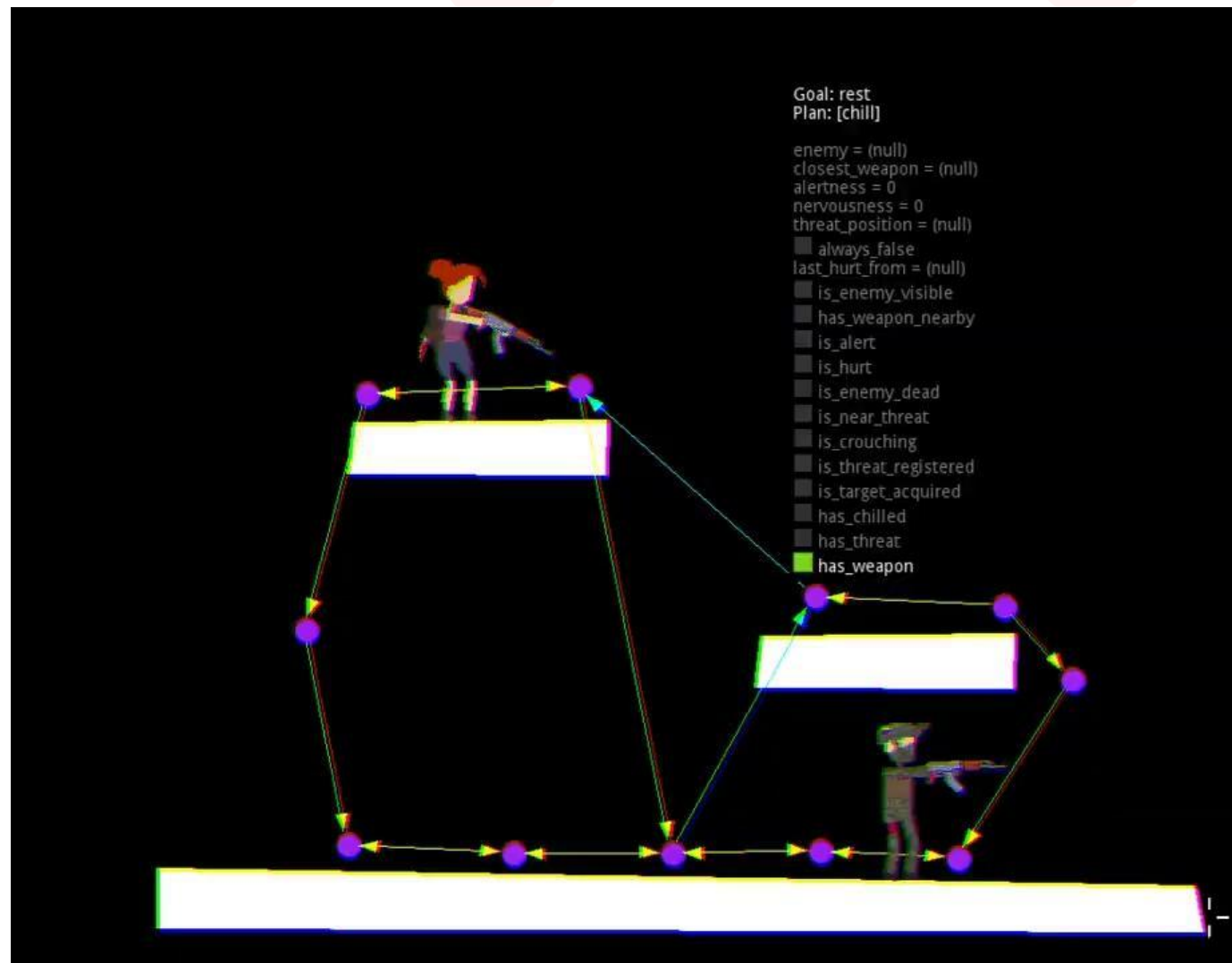


GOAP

In conclusione, il GOAP rimane uno strumento potente e affascinante per il decision-making degli NPC, ma la sua efficacia dipende fortemente dal contesto in cui viene applicato.

La scelta di adottarlo dovrebbe essere basata su un'attenta valutazione delle esigenze di gameplay, delle risorse disponibili e delle capacità del team di sviluppo.

Con una corretta implementazione, il GOAP può offrire NPC memorabili e un gameplay coinvolgente, elevando la qualità complessiva di un videogioco.





Finite State Machine

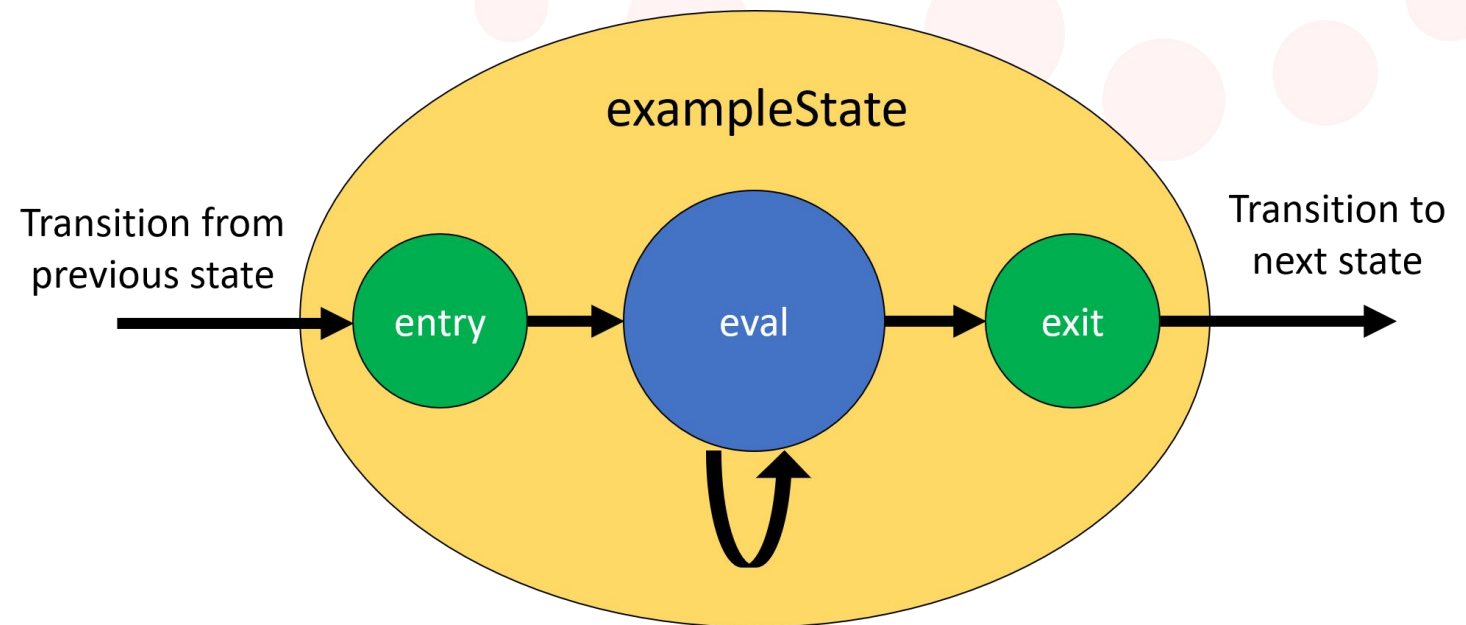
Parlare del sistema *GOAP* ci ha permesso di introdurre il concetto di **stato** all'interno dell'AI. Tuttavia, *GOAP* è una tecnologia relativamente recente: è stata sviluppata nei primi anni 2000 e utilizzata nei videogiochi solo a partire dal 2005 (*F.E.A.R.*).

Se vogliamo davvero capire l'origine del concetto di stato e come questo sia stato applicato nei videogiochi, dobbiamo fare un passo indietro nel tempo.

Il concetto di *stato* esiste da molto prima di *GOAP* ed è alla base di un altro modello ben più antico: le Finite State Machines (**FSM**).

Le FSM hanno radici nella teoria degli automi e sono state formalizzate già nei primi decenni del XX secolo.

I loro utilizzo nei videogiochi risale agli anni '70 e '80, quando venivano impiegate per gestire il comportamento degli NPC nei primi giochi arcade.





Finite State Machine - Esempio





Finite State Machine

Le FSM sono uno dei metodi più semplici e ampiamente utilizzati per implementare il comportamento delle AI nei videogiochi.

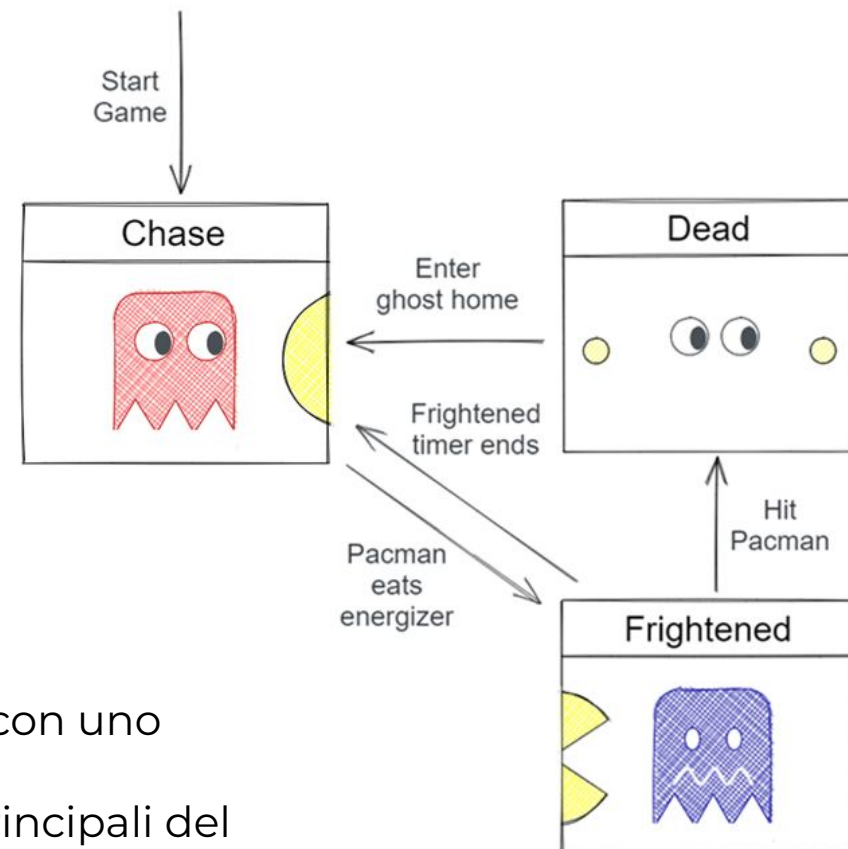
Una FSM è un modello computazionale che rappresenta un'entità in uno stato definito in un determinato momento. L'AI può **transitare** tra stati predefiniti basandosi su **eventi** o **condizioni**, come input del giocatore o cambiamenti nell'ambiente di gioco.

Possiamo descrivere una FSM come un modello astratto di macchina, definita come:

- Un certo numero di stati **S**
- Uno stato di partenza **SO**
- Un certo numero di input **I**
- Un certo numero di transizioni **T** che definiscono il passaggio da uno stato all'altro in base all'input $T = [S1, I, S2]$

Una macchina a stati finiti può essere, quindi, rappresentata con uno **State Diagram**.

Ciò rende più intuitivo ragionare sull'AI ed è uno dei motivi principali del successo di questa tecnica.





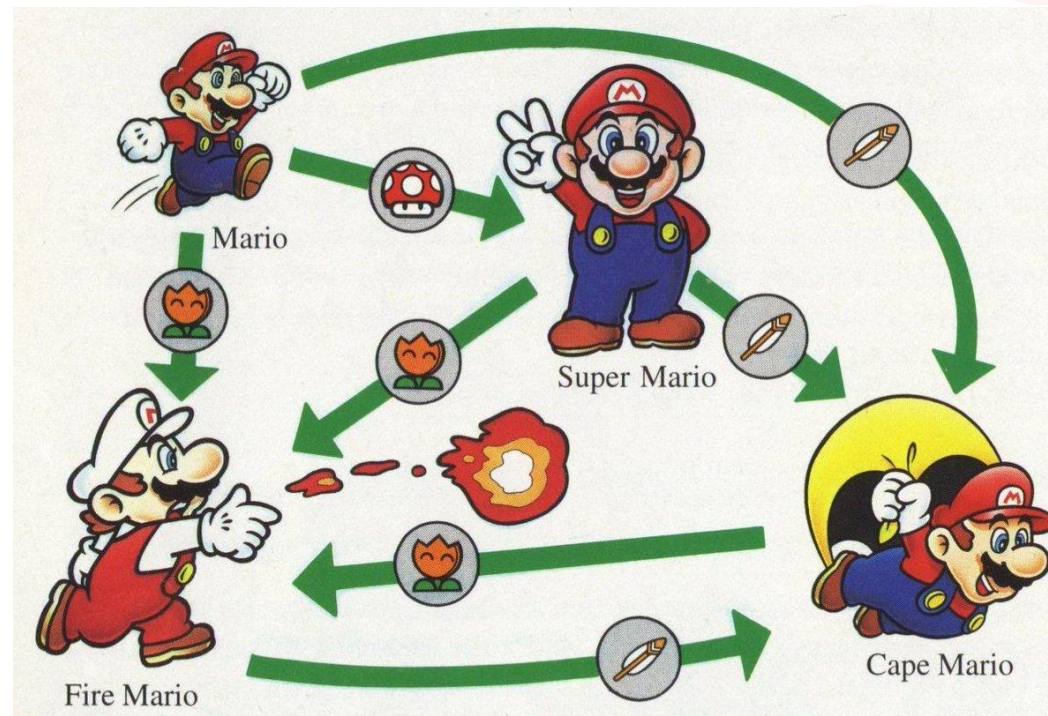
Finite State Machine

Nello sviluppo di un videogioco, possiamo, quindi, rendere più gestibile il comportamento dell'AI separando la propria logica interna da quella esterna: **stato** ed **input**.

Rappresentiamo allora il cervello del personaggio come una serie di stati che reagiscono ad un certo input effettuando delle transizioni tra uno stato e l'altro.

Definiamo inoltre che il personaggio può stare in uno e un solo stato.

Tutto ciò ci permette di rappresentare il comportamento del personaggio tramite una FSM.





Finite State Machine



- **Semplicità:** Sono facili da capire, implementare e debuggare.
- **Determinismo:** Producono comportamenti prevedibili, utili per molti giochi.
- **Efficienza:** Richiedono meno risorse rispetto ad algoritmi più complessi.



- **Rigidità:** Non gestiscono bene comportamenti complessi o dinamici.
- **Crescita esponenziale:** Con l'aumento degli stati e quindi delle transizioni, la complessità del sistema può diventare difficile da gestire.
- **Manutenzione:** Aggiungere, modificare o rimuovere nuovi stati o transizioni può essere complicato e portare a errori.





Fuzzy Logic

Come abbiamo visto, le *Finite State Machines* utilizzano transizioni discrete tra stati ben definiti. Ad esempio, un NPC in un gioco stealth può essere in uno dei seguenti stati:

- Patrolling 🚶
- Investigating 🔍
- Chasing 🏃
- Returning to patrol ↺

Le transizioni tra questi stati sono solitamente determinate da **condizioni binarie**:

- *Se il nemico vede il giocatore → passa a "Chasing".*
- *Se il nemico non vede più il giocatore → passa a "Investigating" o "Returning to patrol".*

Queste transizioni rigide possono risultare poco realistiche, causando comportamenti prevedibili e poco adattivi.

Uno strumento utile per migliorare l'AI, rendendole più naturali e adattive è la **Fuzzy Logic**, ovvero un sistema di ragionamento che permette di gestire situazioni in cui le decisioni non sono rigidamente binarie (*vero o falso*), ma possono avere valori intermedi.





Fuzzy Logic

Con la *Fuzzy Logic* nelle FSM, quindi, le transizioni tra stati non sono più basate su condizioni assolute (*vero o falso*), ma su valori continui tra 0 e 1. Questo consente di gestire stati ibridi e decisioni più sfumate.

Esempio: invece di avere una condizione netta come "*Se il giocatore è a meno di 10 metri → attacca*", possiamo avere:

- Aggressività NPC = funzione dipendente dalla distanza, dalla salute e dall'arma impugnata dal giocatore.
- Transizione graduale tra stati, dove l'NPC può essere parzialmente in *Investigating* e parzialmente in *Chasing*, con percentuali variabili.

A seconda di altre variabili (*visibilità del giocatore, illuminazione, rumore*), il valore di allerta può cambiare progressivamente, evitando cambiamenti bruschi e innaturali nel comportamento dell'NPC.

Distanza dal giocatore	Stato della guardia	Livello di allerta (0-1)
> 20m	Patrolling	0.0
15m - 20m	Investigating	0.3
10m - 15m	Suspicious	0.6
< 10m	Chasing	1.0

Ovviamente, in questo modo, aumenta drasticamente la complessità di implementazione e bilanciamento delle FSM così come le risorse computazionali richieste.





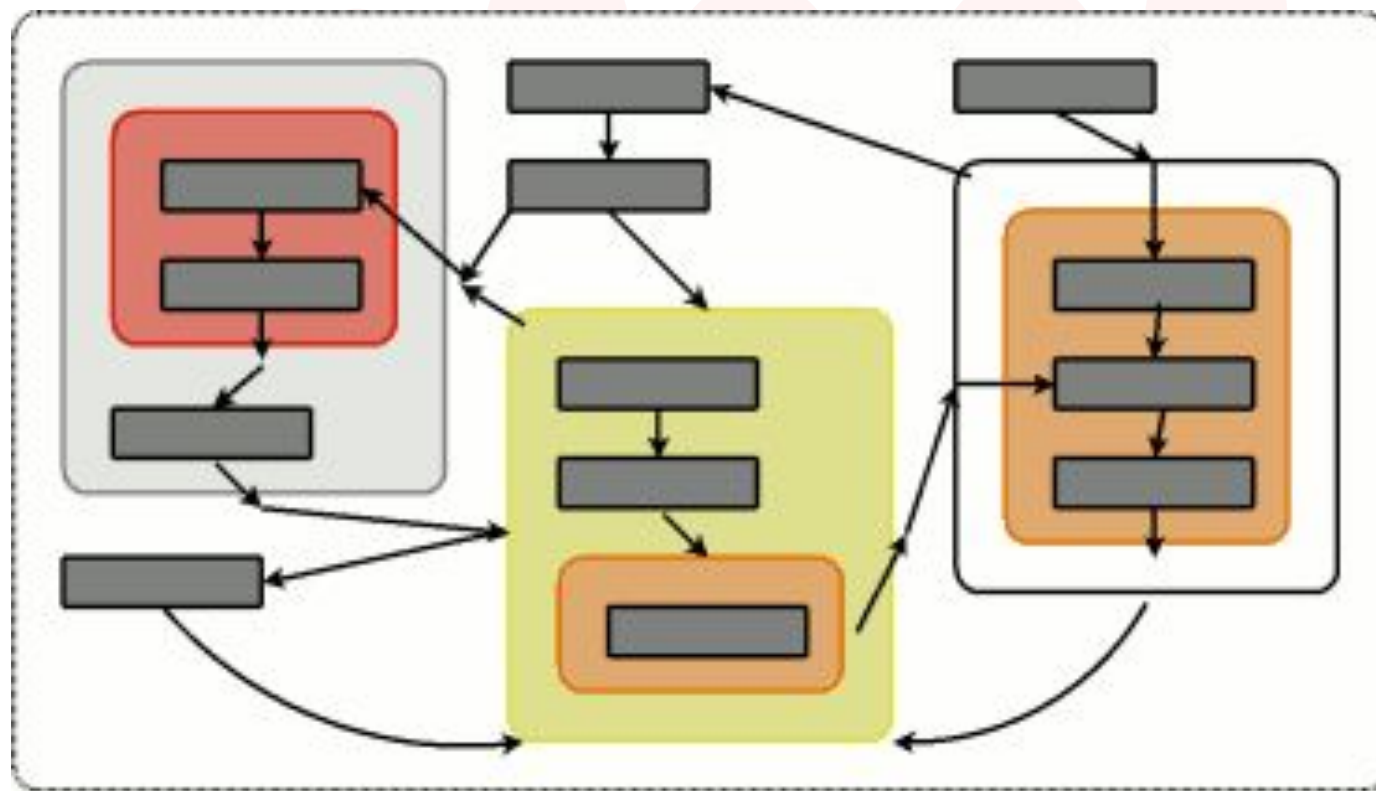
Hierarchical FSM

In una FSM classica, ogni stato è indipendente e la macchina può trovarsi in un solo stato alla volta.

Tuttavia, tramite le *Hierarchical Finite State Machines* (**HFSM**), un'estensione delle FSM che introduce il concetto di **gerarchia**, gli stati possono essere organizzati in una struttura ad albero, dove uno stato "genitore" può contenere sotto-stati "figli".

Questo permette di:

- Evitare la duplicazione del codice, poiché stati simili possono ereditare comportamenti comuni.
- Gestire comportamenti più complessi, suddividendo le logiche in sottogruppi.
- Facilitare le transizioni, consentendo cambi di stato a livello superiore senza perdere il contesto dei sotto-stati.





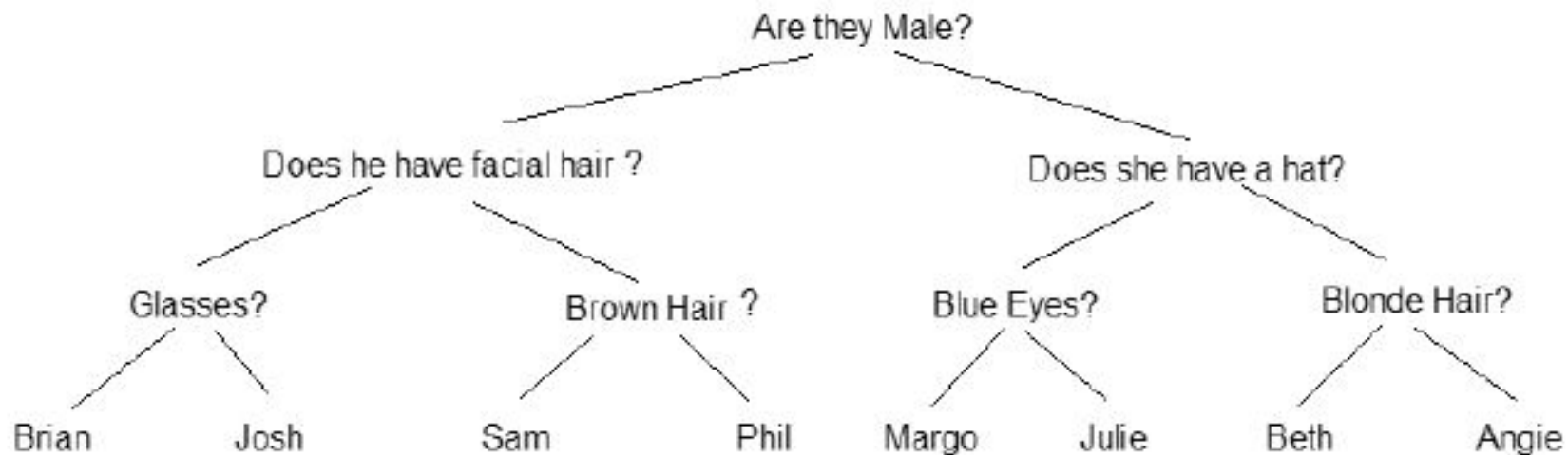
Alberi Decisionali

Con l'aumento della complessità dell'AI nei videogiochi, una nuova soluzione si è ottenuta con l'utilizzo di Alberi Decisionali, ovvero una struttura gerarchica in cui **ogni nodo rappresenta una domanda** o condizione, e **i rami rappresentano le possibili risposte**.

L'AI segue un percorso dall'altro verso il basso fino a raggiungere un **nodo foglia**, che determina l'azione finale.

Questo strumento è un approccio alternativo basato su condizioni, utile per prendere decisioni logiche in base a più variabili.

Albero decisionale binario per una partita di «Indovina Chi?»





Alberi Decisionali

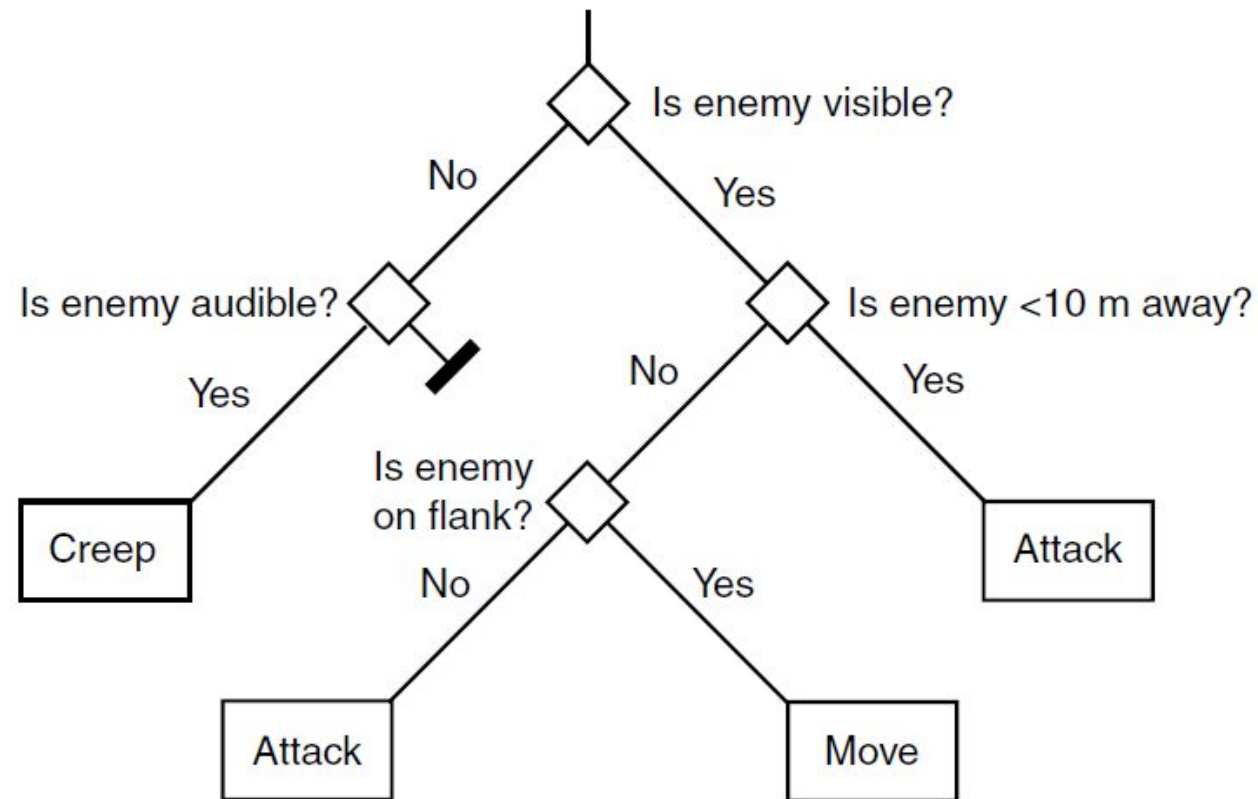
Possiamo progettare l'AI decisionale di un personaggio tramite un albero binario:

- **Decision / Test: incroci dei rami**

Definiscono i ragionamenti che l'AI compie sulle variabili interne ed esterne

- **Azioni: foglie**

Definiscono quale azione compierà l'AI in base ai ragionamenti





RECAP



Nel contesto dello sviluppo dell'AI nei videogiochi, le **FSM** sono state per anni una soluzione diffusa per modellare il comportamento. Tuttavia, come abbiamo visto, queste architetture presentano limiti evidenti: la gestione di stati complessi porta rapidamente a una crescita esponenziale delle transizioni, rendendo difficile la manutenzione e l'estendibilità del sistema.

Anche le loro varianti gerarchiche, le **HFSM** nonostante aiutino ad avere una migliore organizzazione delle transizioni, rimuovendo, anche, molta ripetizione di codice, mantengono una debolezza data dal fatto che, in ogni sottogruppo di una gerarchia, potrebbero esserci nodi che sono sostanzialmente una riscrittura di nodi appartenenti a un sottogruppo diverso, poiché i nodi in ciascuna gerarchia non sono ancora modulari e restano strettamente legati alla loro gerarchia.

Gli **alberi decisionali** offrono un'alternativa strutturata, valutando ogni volta un percorso dalla radice alla foglia in base a condizioni predefinite. Sebbene migliorino l'organizzazione rispetto alle FSM, soffrono anch'essi di rigidità: ogni decisione è valutata interamente a ogni esecuzione, senza possibilità di persistenza dello stato intermedio, con un costo computazionale crescente all'aumentare della complessità.

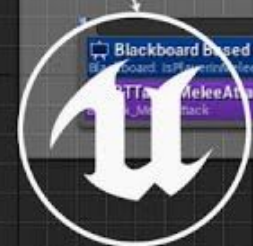
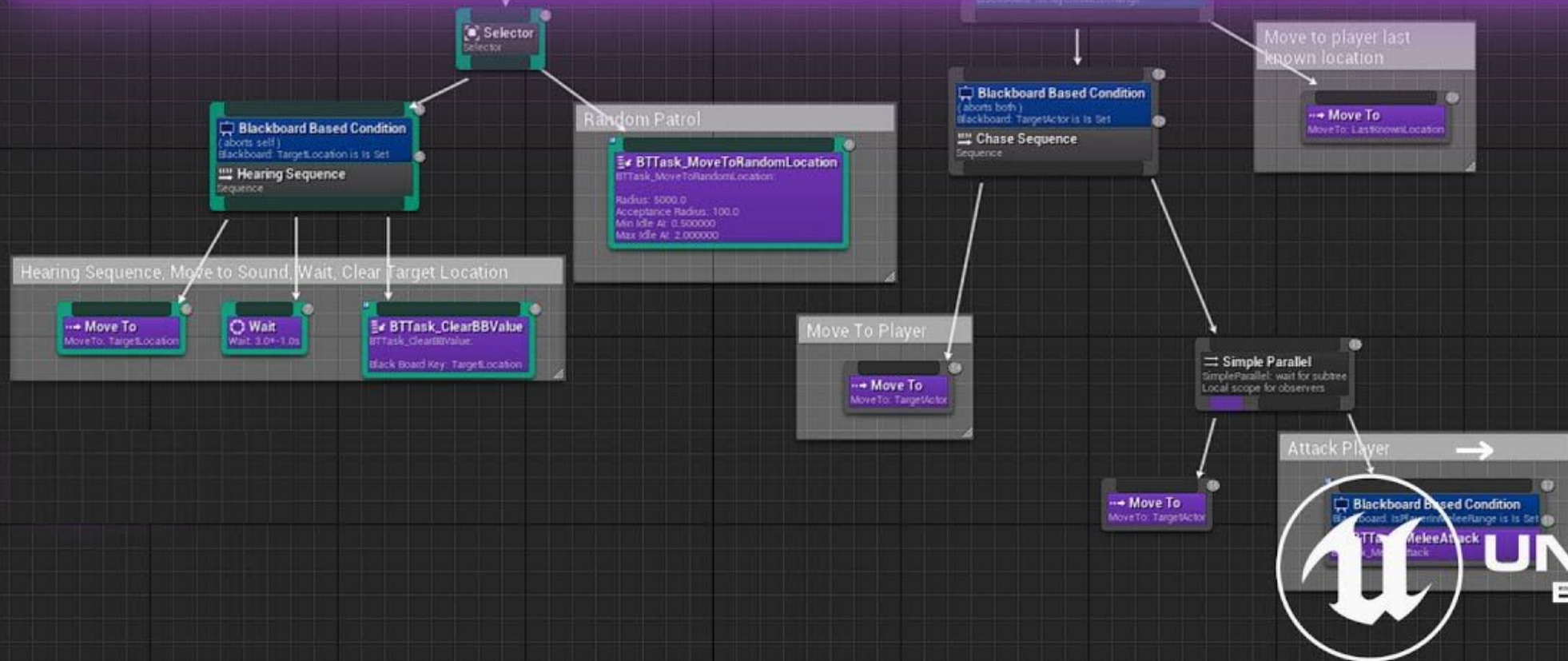
Per superare questi limiti, si è trovata una soluzione più flessibile e modulare.

A differenza degli alberi decisionali, in questa soluzione mantiene lo stato tra un aggiornamento e l'altro, permettendo di riprendere l'esecuzione da un nodo in esecuzione senza dover rivalutare l'intero albero. Inoltre, la sua modularità consente di riutilizzare parti dell'albero in diversi contesti, favorendo la scalabilità del comportamento degli NPC.

Questa struttura offre quindi un equilibrio tra controllo, modularità e flessibilità, rendendo così i **Behaviour Tree** una scelta potente per gestire comportamenti complessi in ambienti dinamici.



BEHAVIORS TREE



UNREAL
ENGINE

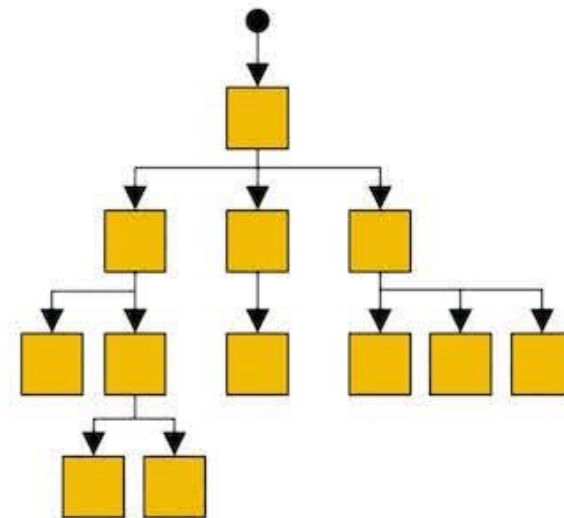
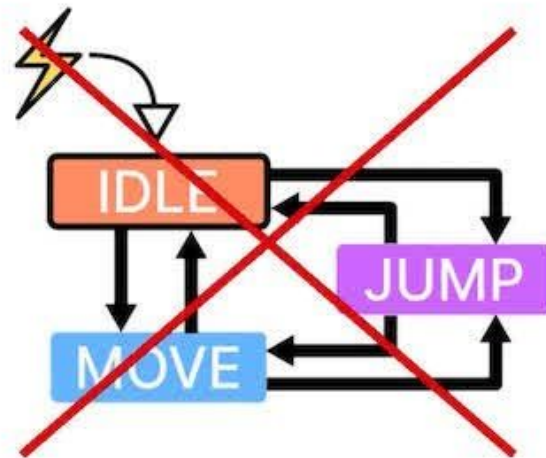


Behaviour Tree

Nell'universo dello sviluppo di videogiochi, i Behaviour Tree sono strutture gerarchiche che governano i processi decisionali dei personaggi AI, determinando le loro azioni e reazioni durante il gameplay.

Per un programmatore di giochi, approfondire le complessità dei Behaviour Tree è fondamentale, poiché permette di creare entità virtuali dinamiche, intelligenti e coinvolgenti, migliorando così l'esperienza di gioco del giocatore.

Sebbene i Behaviour Tree condividano alcune somiglianze con le *HFSM* – come l'organizzazione in gerarchie che favorisce il riutilizzo degli elementi – la distinzione principale è che nei Behaviour Tree gli elementi fondamentali sono i **Task**, non gli Stati. Il loro principale vantaggio è la natura intuitiva, che li rende meno soggetti a errori; per questo motivo, sono ampiamente adottati nell'industria dello sviluppo di videogiochi.





Behaviour Tree

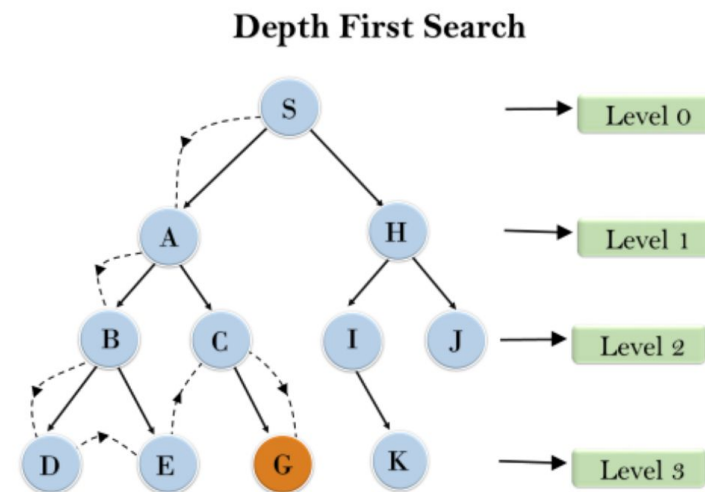
I videogiochi moderni sono sempre più complessi, e ciò comporta una crescita proporzionale della complessità dei personaggi AI. Di conseguenza, la manutenibilità di questi personaggi – o agenti – diventa cruciale.

A differenza di sistemi come le macchine a stati finiti, che diventano difficili da gestire all'aumentare del numero di stati, i Behaviour Tree offrono una soluzione pratica e scalabile per i processi decisionali.

I principali vantaggi dei Behaviour Tree rispetto ad altri sistemi risiedono nella loro **scalabilità**, **espressività** ed **estensibilità**. A differenza di altri approcci, i Behaviour Tree non prevedono transizioni esplicite tra stati; ogni nodo dell'albero specifica invece come eseguire i propri figli. Questa natura priva di stato elimina la necessità di tracciare i nodi eseguiti in precedenza per determinare il prossimo comportamento. L'espressività dei Behaviour Tree deriva dall'uso di vari livelli di astrazione, transizioni implicite e strutture di controllo avanzate per i nodi compositi.

Quando un agente esegue un Behaviour Tree, effettua una ricerca in profondità (**depth-first search**) per individuare ed eseguire il nodo foglia di livello più basso.

Inoltre, nei Behaviour Tree le transizioni avvengono tramite chiamate e valori di ritorno scambiati tra i nodi dell'albero, facilitando un meccanismo di trasferimento del controllo bidirezionale.

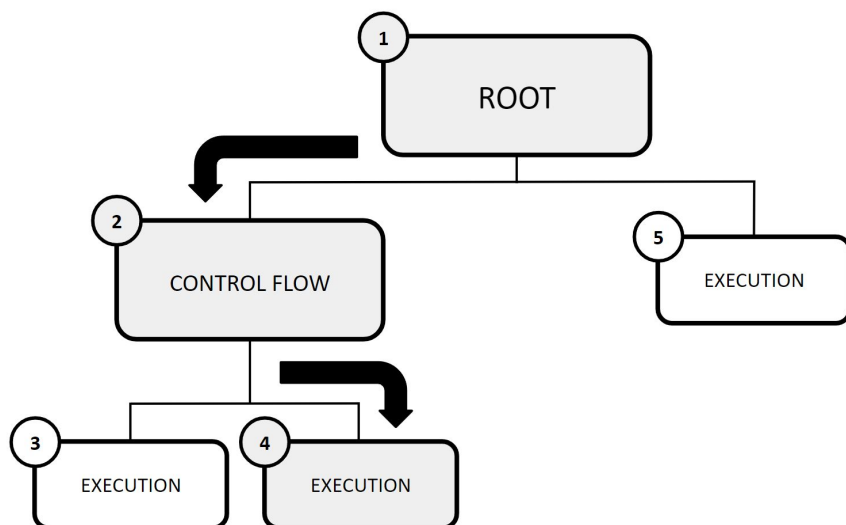




Behaviour Tree

Un Behaviour Tree viene rappresentato visivamente come una struttura ad albero, dove ogni nodo può avere un nodo genitore e uno o più figli ed in cui i nodi sono classificati in tre categorie:

- Il nodo radice (**Root**): non ha genitori e possiede un solo figlio.
- I nodi di controllo del flusso (**Control Flow**): hanno un genitore e almeno un figlio.
- I nodi di esecuzione (**Execution**): hanno un genitore ma nessun figlio.



- 1) L'esecuzione di un Behaviour Tree inizia dal nodo radice, che invia segnali di esecuzione ai suoi nodi figli.
- 2) Quando si raggiunge un nodo di controllo del flusso, esso gestisce l'esecuzione e il flusso decisionale dell'albero, determinando quali compiti o sotto-alberi devono essere eseguiti in base a specifiche condizioni o regole.

- 3) Ogni volta che viene attivato un nodo di esecuzione, esso esegue un compito specifico e comunica il proprio stato al nodo genitore, restituendo uno dei seguenti risultati:

- **Running** se il compito è ancora in corso.
- **Success** se l'operazione è stata completata con successo.
- **Failure** se il compito non è andato a buon fine.

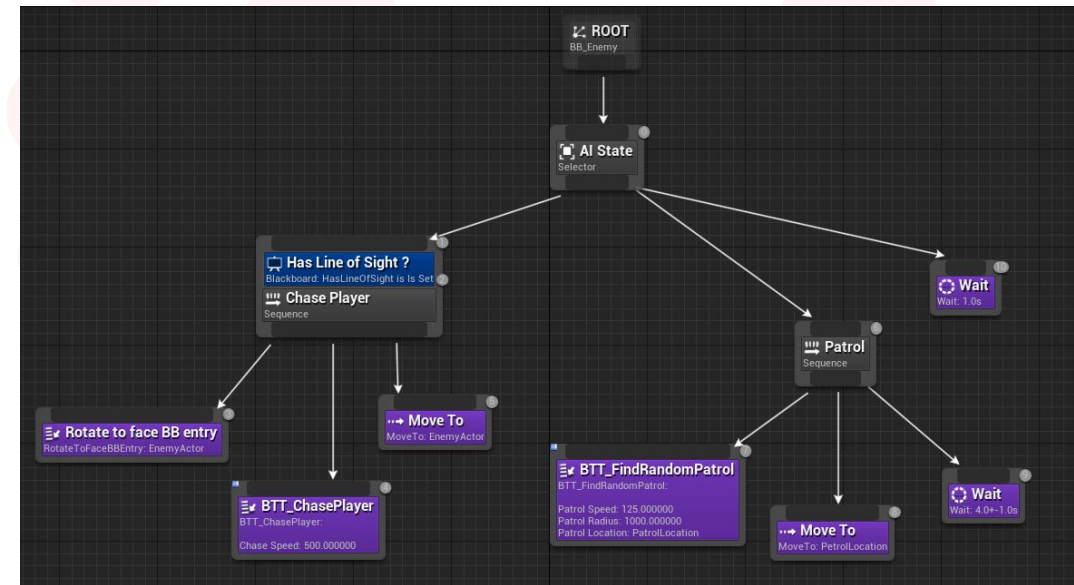


Behaviour Tree

In *Unreal Engine*, i Behaviour Tree (**BT**) sono asset che vengono modificati in modo simile ai *Blueprint*, ovvero attraverso un'interfaccia visiva, aggiungendo e collegando un insieme di nodi con funzionalità specifiche per formare un grafo comportamentale.

L'esecuzione di un BT è gestita da un istanza di **BehaviorTreeComponent**, che è contenuta all'interno di un'istanza di **AIController**.

Tuttavia, questo componente non viene automaticamente collegato al controller e deve essere aggiunto manualmente tramite *C++* o *Blueprint*. Se nessun componente è presente, verrà creato automaticamente a runtime.



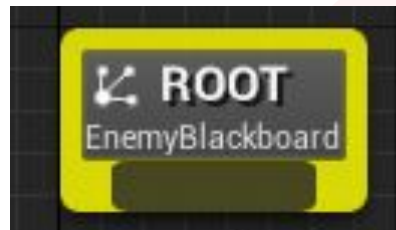
Un aspetto chiave che distingue i Behaviour Tree di Unreal Engine da altri sistemi è la loro natura **event-driven**, che evita l'esecuzione continua del codice.

Invece di controllare costantemente i cambiamenti rilevanti, un Behaviour Tree in Unreal Engine ascolta specifici eventi che possono determinare modifiche nell'albero. Questo approccio basato su eventi offre miglioramenti in termini di prestazioni e vantaggi nelle capacità di debugging.

Vediamo ora la come sono classificati gli elementi di un BT in UE →



Root Node



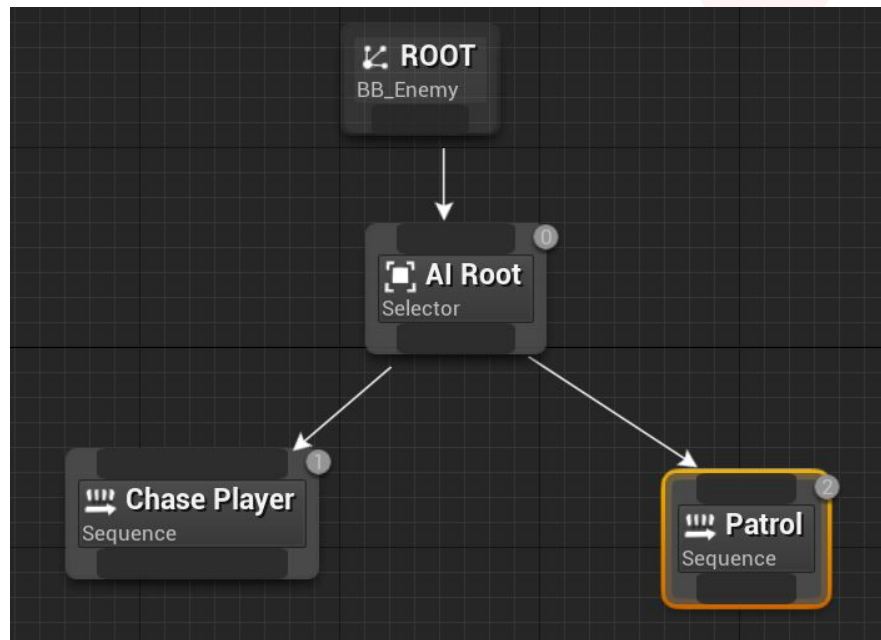
Il nodo radice (**Root**) funge da punto di partenza per un Behaviour Tree.

Occupa una posizione unica all'interno della struttura e segue un insieme di regole specifiche:

- Può esserci un solo nodo radice all'interno dell'albero.
- Può avere una sola connessione; se questa viene rimossa, l'intero Behaviour Tree viene disattivato.
- Non supporta l'aggancio di nodi *Decorator* o *Service*.



Composite Nodes



I **nodi Composite** definiscono la radice di un ramo all'interno del Behaviour Tree e stabiliscono le regole per la sua esecuzione. Inoltre, sono gli unici nodi che possono essere collegati direttamente al nodo radice di un Behaviour Tree.

Un nodo Composite può avere *Decorator* e *Service* applicati, permettendo di integrare logiche più complesse. Quando un *Service* viene applicato a un nodo Composite, esso rimane attivo per tutta la durata dell'esecuzione dei nodi figli.



Composite Nodes

Esistono tre nodi Composite disponibili:

1. I nodi **Selector** eseguono i loro figli sequenzialmente da sinistra a destra e interrompono l'esecuzione non appena uno di essi ha successo.

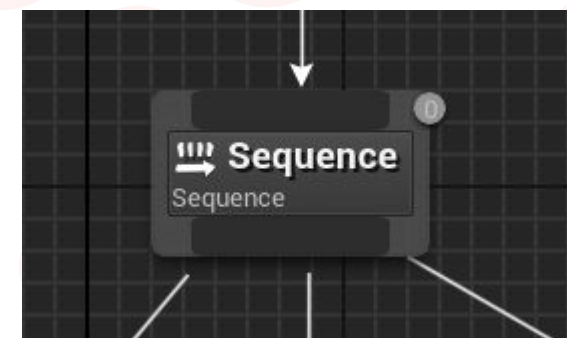
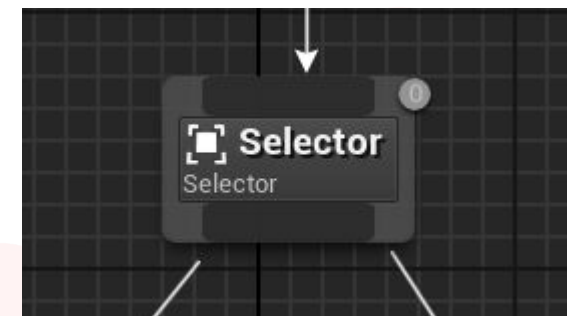
Se un figlio di un nodo *Selector* restituisce successo, allora anche il *Selector* viene considerato riuscito. Al contrario, se tutti i figli di un *Selector* falliscono, il nodo stesso viene contrassegnato come fallito.

2. I nodi **Sequence** eseguono i loro figli sequenzialmente da sinistra a destra e interrompono l'esecuzione quando uno dei figli fallisce.

Se un figlio fallisce, l'intera sequenza fallisce. A differenza dei *Selector*, il successo di una *Sequence* si ottiene solo quando tutti i suoi figli hanno successo.

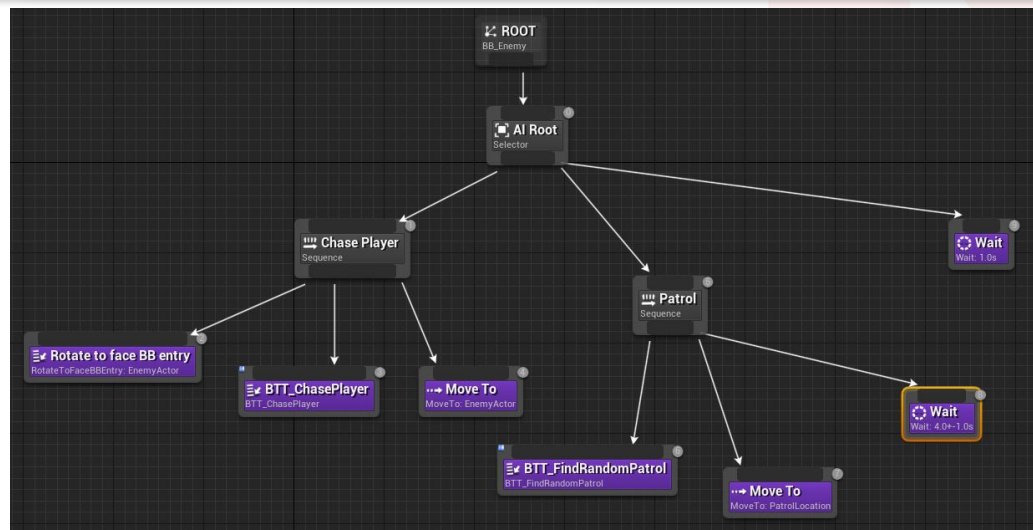
3. I nodi **Simple Parallel** consentono l'esecuzione di un singolo nodo di task principale in parallelo con l'intero albero.

Dopo che il task principale è stato completato, puoi decidere – tramite l'attributo *Finish Mode* – se il nodo deve terminare immediatamente, fermando l'albero secondario, oppure se deve attendere che l'albero secondario termini prima di completarsi.





Task Nodes

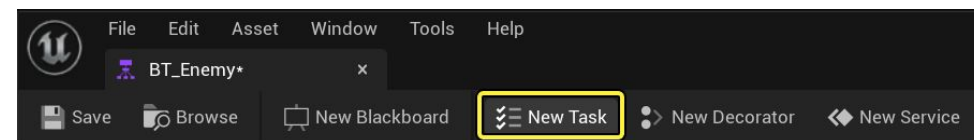


I **node Task** sono responsabili dell'esecuzione di azioni e rappresentano le singole operazioni che un agente AI può eseguire. Possono essere utilizzati per creare azioni semplici o combinati tra loro per sviluppare comportamenti più articolati, ma non interrompono la propria esecuzione fino a quando non restituiscono un risultato di successo o fallimento.

Un nodo Task può avere uno o più *Decorator* o *Service* collegati, permettendo di creare comportamenti più complessi e interazioni avanzate all'interno dell'ambiente di gioco.

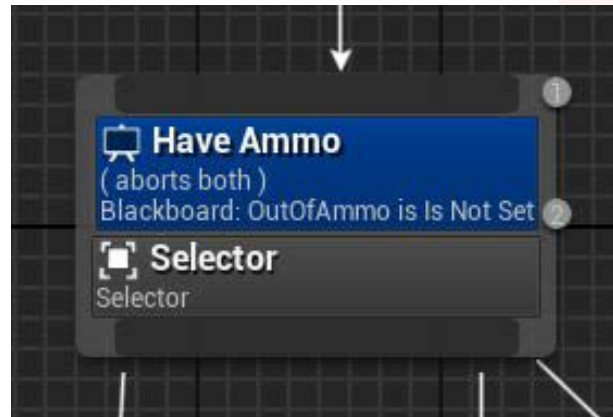
UE fornisce un insieme di Task predefiniti, pronti all'uso, che coprono i casi d'uso più comuni per gli sviluppatori.

Tuttavia, è possibile estendere i Task per creare nodi personalizzati in base alle esigenze specifiche del progetto.





Decorator Nodes



I **Decorators** vengono utilizzati per determinare se un ramo o un singolo nodo può essere eseguito. Devono essere attaccati a un nodo composite o a un nodo task.

I Decorators giocano un ruolo cruciale nel determinare il percorso di esecuzione dei rami all'interno del Behaviour Tree; agiscono fondamentalmente come decisori, valutando se un ramo specifico o un nodo individuale dovrebbe proseguire nell'esecuzione. Essi funzionano come una condizione, valutando la fattibilità di proseguire lungo un determinato ramo e segnalando un fallimento preventivo se il task (o un sotto-albero) è destinato a fallire.

Questa azione pre-emptiva aiuta a evitare che il decorator tenti di eseguire un task (o un sotto-albero) che è destinato a fallire per vari motivi, come la mancanza di informazioni o obiettivi obsoleti.

UE include un set di Decorators predefiniti pronti all'uso, ma è possibile estenderli per crearne di personalizzati. La maggior parte di questi include una proprietà, **Inverse Condition**, che consente di invertire la condizione, offrendo maggior flessibilità nella gestione delle logiche di esecuzione.



Service Nodes



I **Service** non eseguono azioni dirette, come i *Task*, ma si concentrano principalmente su attività di monitoraggio o aggiornamento, dell'AI o del suo BT, che devono essere effettuate periodicamente.

Possono essere attaccati a nodi *Composite* o *Task* e vengono eseguiti a intervalli specifici, definiti nell'attributo **Interval**, mentre il loro ramo è attivo.

Una volta attivato, un *Service* continua ad essere eseguito indipendentemente dal numero di livelli genitore-figlio in esecuzione sotto il nodo che lo possiede.

Rispetto ai *Decorator*, i *Service* sono dei nodi molto specifici rispetto al Behaviour Tree che si va a sviluppare; ciò significa che, molto probabilmente, se ne dovranno creare di personalizzati in base ad ogni AI o progetto.



Blackboard

È importante notare che i BT esistono come oggetti condivisi (**shared objects**) all'interno del progetto. Questo significa che tutti le AI che utilizzano un determinato BT condivideranno la stessa istanza e, di conseguenza, gli oggetti condivisi non potranno memorizzare dati specifici per ogni agente. I principali vantaggi di questo approccio sono migliori prestazioni della CPU e minore utilizzo della memoria.

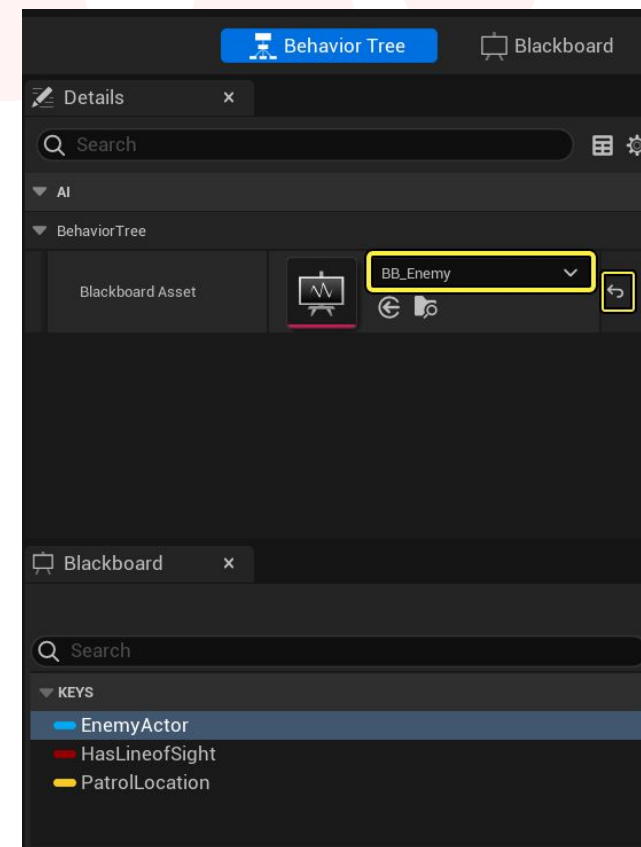
Tuttavia, è possibile gestire dati specifici per ogni agente in diversi modi. Uno di questi è l'uso del **Blackboard**, il quale offre maggiore flessibilità nell'utilizzo del Behaviour Tree.

La *Blackboard* è una componente fondamentale dei Behaviour Tree; agisce come una sorta di **spazio di memoria** – una sorta di *cervello* – in cui le AI possono leggere e scrivere dati durante il loro processo decisionale. Ciò significa che gli sviluppatori possono interrogare e aggiornare le informazioni memorizzate al suo interno.

La Blackboard viene creata come un asset di tipo **Blackboard Data**, che verrà assegnato a un Behaviour Tree.

Contiene un set di variabili – chiamate **keys** – che memorizzano informazioni di tipi predefiniti.

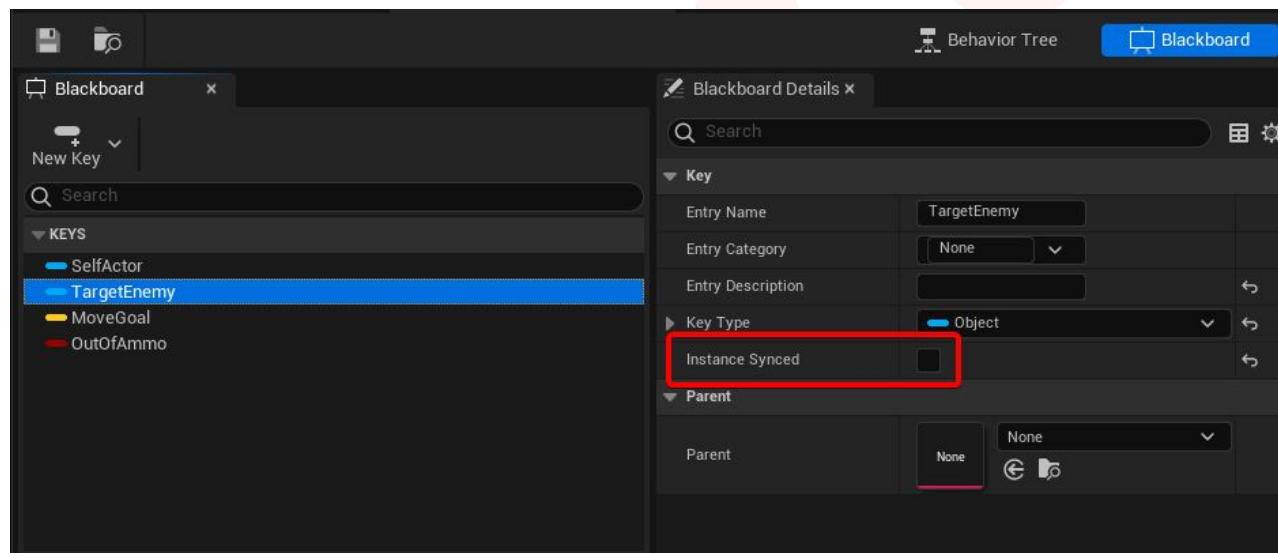
Queste chiavi possono essere accedute e modificate durante l'esecuzione per influenzare il processo decisionale dei personaggi AI.





Blackboard

Una key può essere impostata su **Instance Synced**; in questo caso, la chiave stessa sarà sincronizzata tra tutte le istanze del *Blackboard*. Questa sincronizzazione garantisce che qualsiasi modifica apportata al valore della chiave venga riflessa in modo consistente su tutte le istanze delle AI che condividono lo stesso Behaviour Tree e Blackboard.



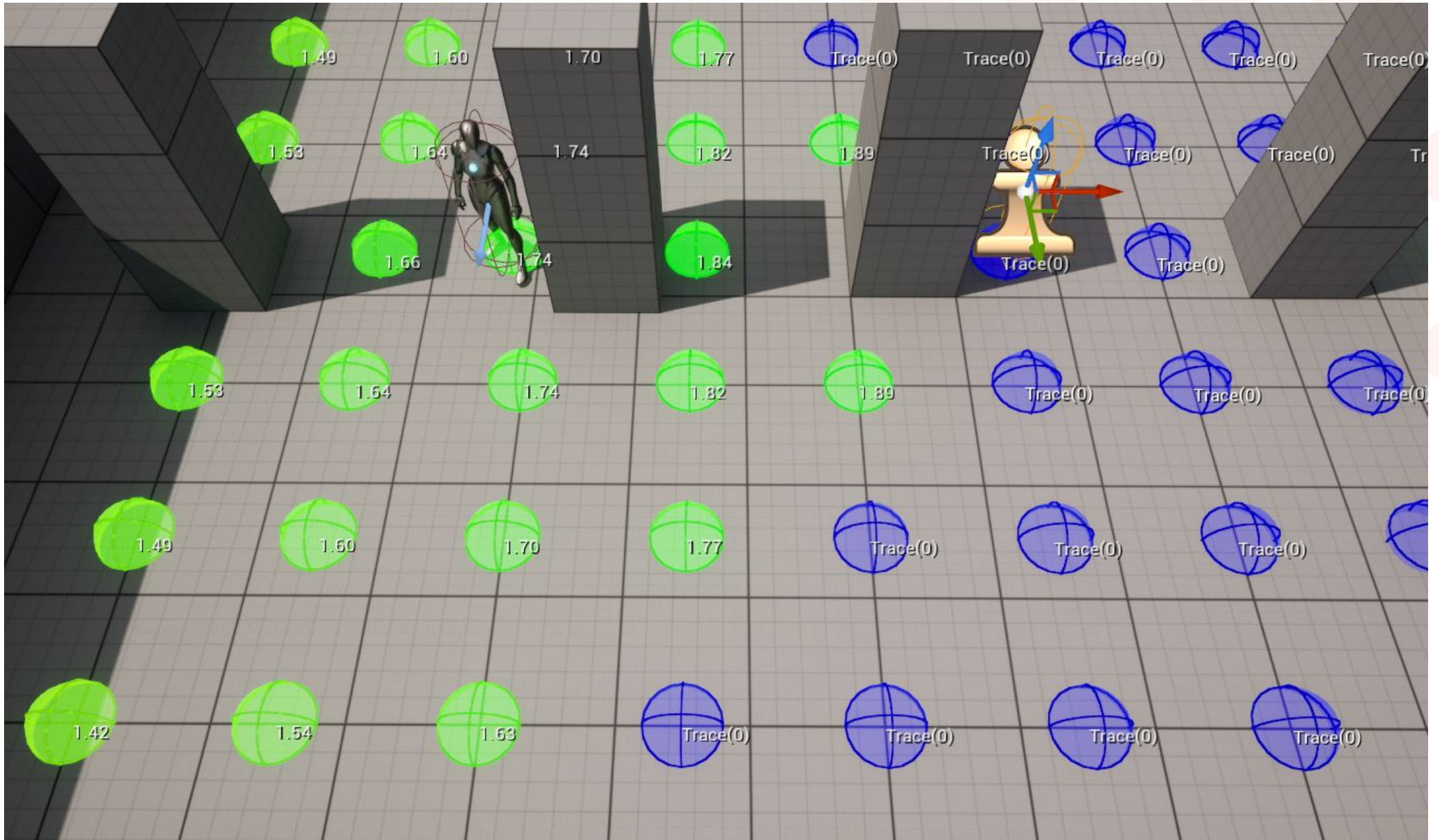
L'istanza di **Blackboard Component** permette di interrogare i dati della Blackboard e di memorizzare dati nella Blackboard stessa.

Una Blackboard può memorizzare fino a 255 chiavi e supporta diversi tipi di dati come *FVector*, *FRotator*, *Bool*, *Int32*, *Float*, *UClass*, *UObject*, *FName*, *UEnum*, *FString*, ma non può memorizzare gli arrays.

Nonostante la sua apparente semplicità, comprendere come funziona un Blackboard è cruciale per garantire il corretto funzionamento degli AI agents nel tuo progetto.



Environment Query System





Environment Query System

L'*Environment Query System* (**EQS**) in Unreal Engine è una potente funzionalità all'interno del framework AI che consente agli sviluppatori di raccogliere dati sull'ambiente virtuale, permettendo agli agenti AI di interrogarlo e prendere decisioni informate in base ai risultati ottenuti.

Padroneggiando l'EQS, si avrà il potere di creare sistemi AI intelligenti in grado di prendere decisioni informate in base all'ambiente circostante. Che si tratti di trovare il miglior punto di osservazione, localizzare risorse cruciali o pianificare strategie per un gameplay ottimale, l'EQS apre un mondo di possibilità. Sebbene sia ancora una funzionalità sperimentale, imparare a utilizzarlo ti permetterà di sviluppare sistemi AI intelligenti e dinamici in Unreal Engine.

L'EQS permette agli sviluppatori di creare comportamenti AI che si adattano dinamicamente alle condizioni ambientali in continua evoluzione.

Infatti, grazie all'EQS, gli NPC e altre entità di gioco possono prendere decisioni intelligenti basate sul loro ambiente.

Con la sua flessibilità e facilità d'uso, l'EQS è una funzionalità preziosa di UE, ideale per creare esperienze di gameplay immersive e interattive.

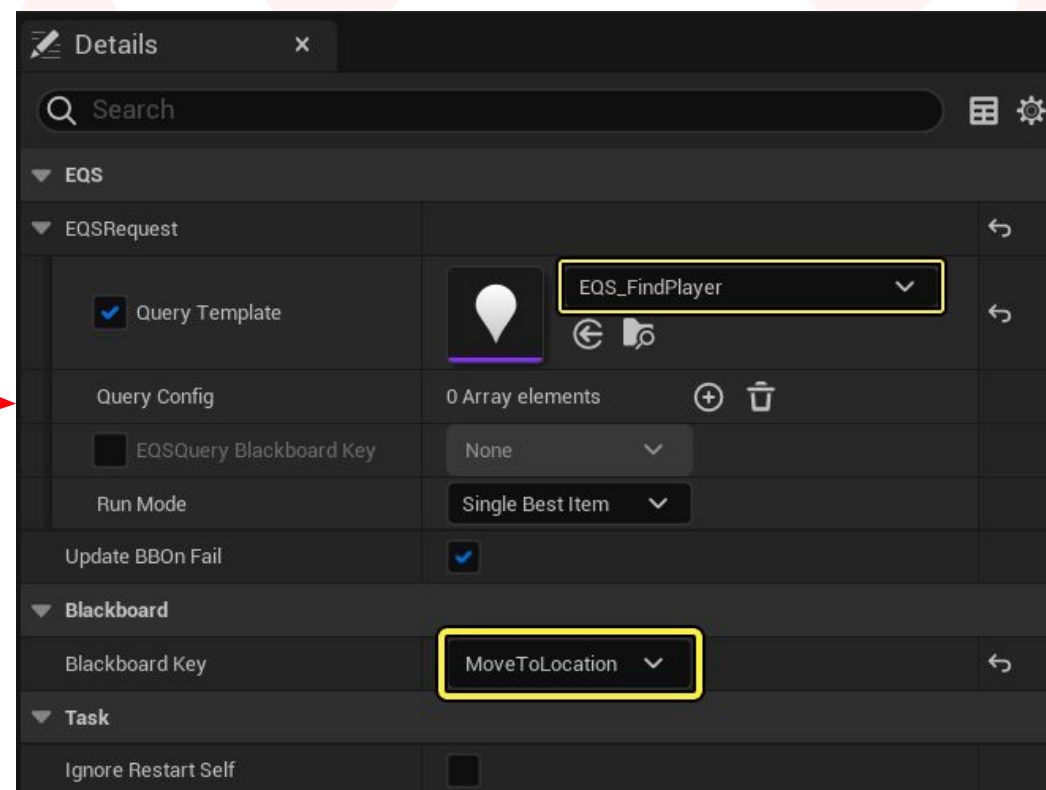
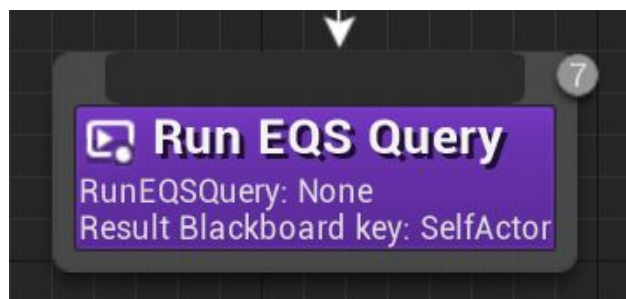




Environment Query System

L'EQS consente di analizzare i dati raccolti attraverso una serie di **Test** che generano elementi più coerenti con la natura della query effettuata.

Una *query EQS* può essere attivata all'interno di un *Behavior Tree* o, in alternativa, tramite scripting, per guidare le decisioni sulla base dei risultati dei test.



Queste query sono costituite principalmente da:

- **Generators:** elementi che determinano le posizioni o gli attori da testare e ponderare.
- **Contexts:** elementi che forniscono un riferimento per i test o i generatori.

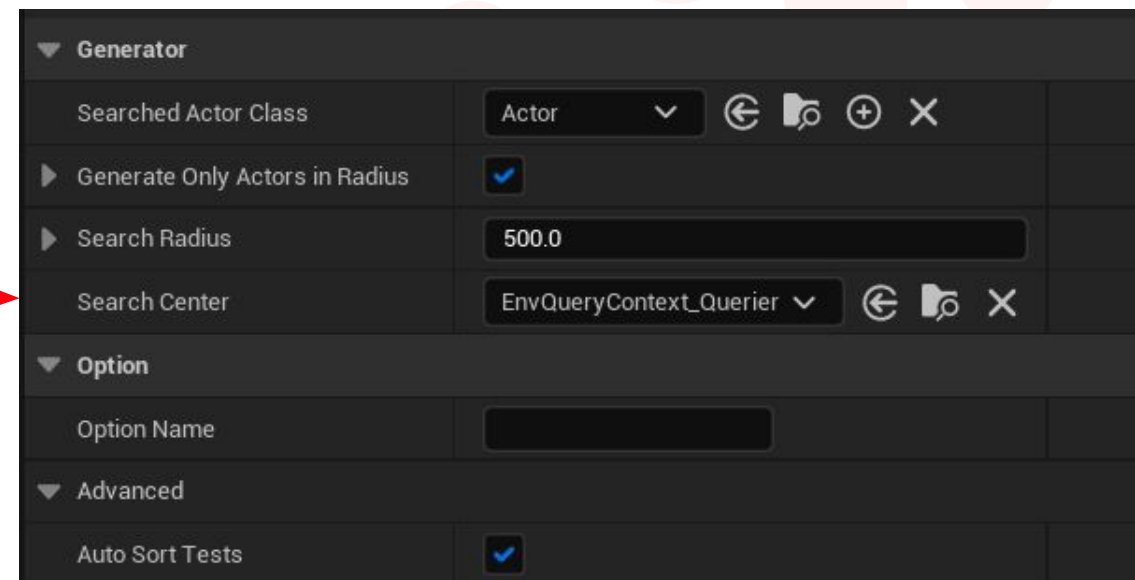
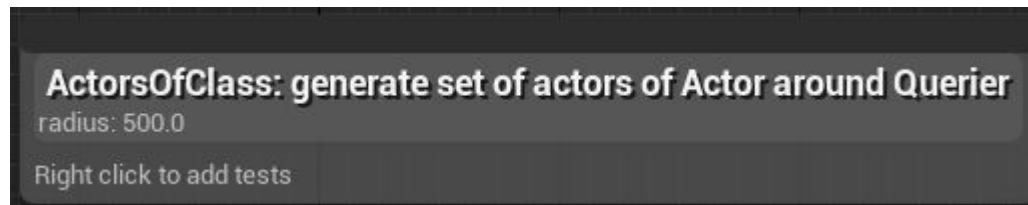


EQS - Generators

Un **Generator** crea le posizioni o gli attori – noti come **items** – che verranno sottoposti a *test* e ponderazione (*pesi*); il risultato verrà restituito al Behavior Tree a cui appartiene la query.

I generatori disponibili out of the box includono:

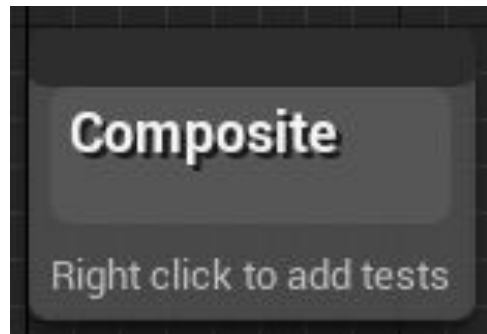
- **Actors of Class:** Trova tutti gli attori di una determinata classe e li restituisce come items per i test.





EQS - Generators

- **Composite:** Permette di creare un array di generatori e utilizzarli per i test.

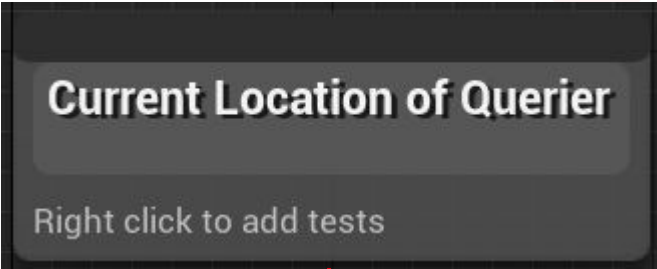


▼ Generator	
Generators	0 Array elements + 🗑️
▼ Advanced	
Allow Different Item Types	☐
Forced Item Type	None ▼ ↶ 🔍 ✕
▼ Option	
Option Name	
▼ Advanced	
Auto Sort Tests	☒



EQS - Generators

- **Current Location:** Ottiene la posizione di un contesto specificato e la utilizza per convalidare i test.



▼ Generator

Query Context

EnvQueryContext_Querier ▼ ↶ 🔍 ✕

▼ Option

Option Name

▼ Advanced

Auto Sort Tests

☒



EQS - Generators

- **Points:** Genera punti basati su forme geometriche, come *Cerchio*, *Cono*, *Donut*, *Griglia* e *Pathing Grid*, attorno a una posizione predefinita.

OnCircle: generate items on circle around Querier

radius: 1000.0, item span: 50.0
navmesh trace

Right click to add tests

Cone: generate in front of Querier

degrees: 90.0, angle step: 10.0, projection (height: 1,024)

Right click to add tests

Donut: generate items around Querier

radius: 300.0 to 1000.0
rings: 3, points per ring: 8

Right click to add tests

SimpleGrid: generate around Querier

radius: 500.0, space between: 100.0, projection (height: 1,024)

Right click to add tests

PathingGrid: generate around Querier

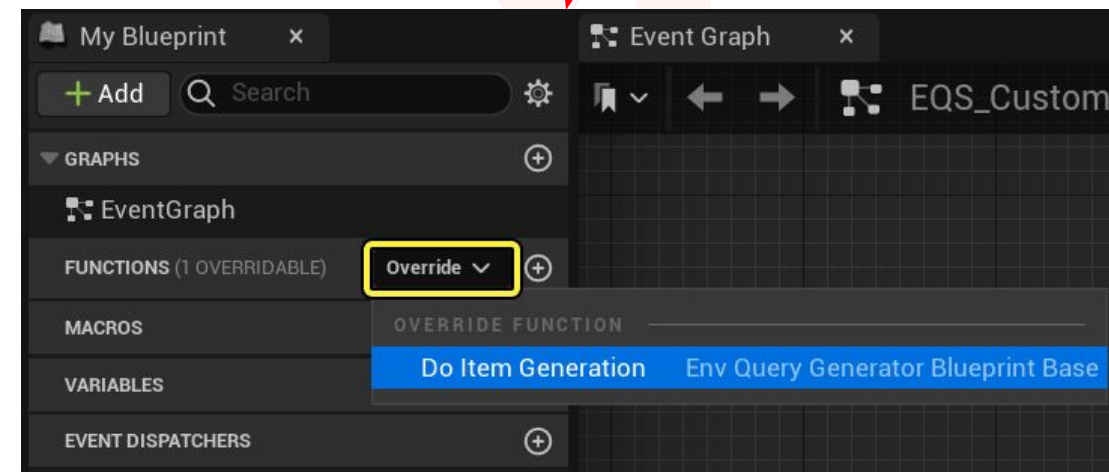
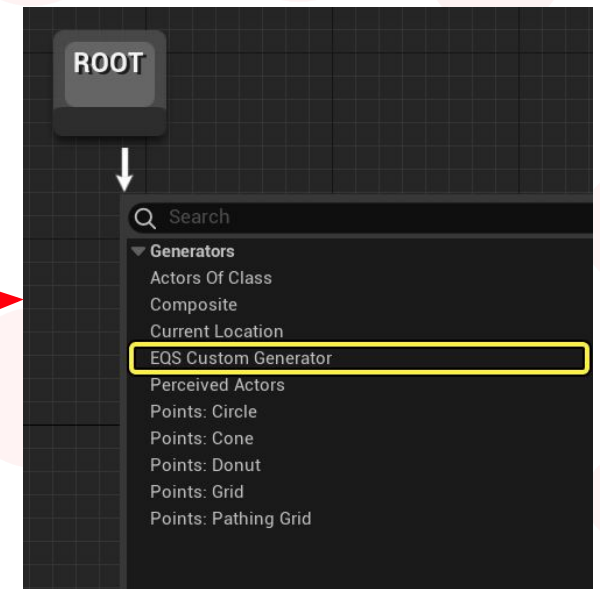
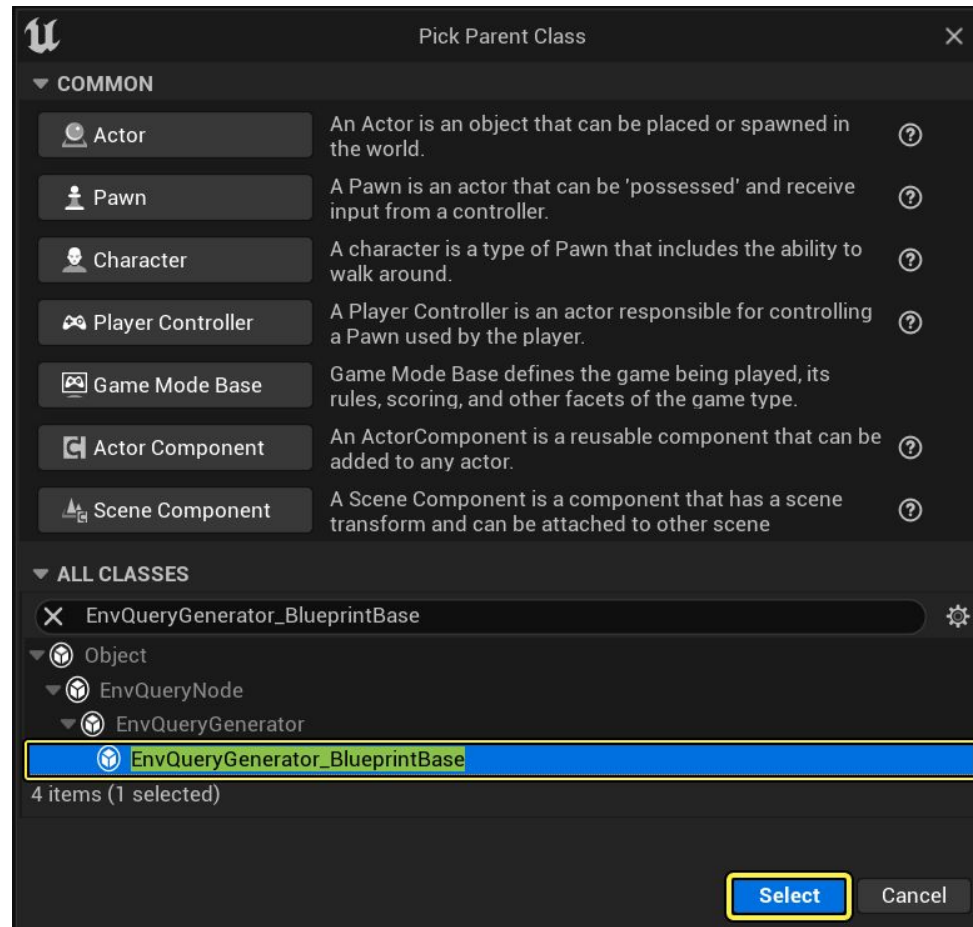
radius: 1000.0, space between: 100.0, projection (height: 1,024)

Right click to add tests



EQS - Generators

Inoltre, puoi implementare i tuoi generatori personalizzati estendendo la classe *EnvQueryGenerator* (se sviluppi in C++) o *EnvQueryGenerator_BlueprintBase* (se sviluppi con Blueprints).





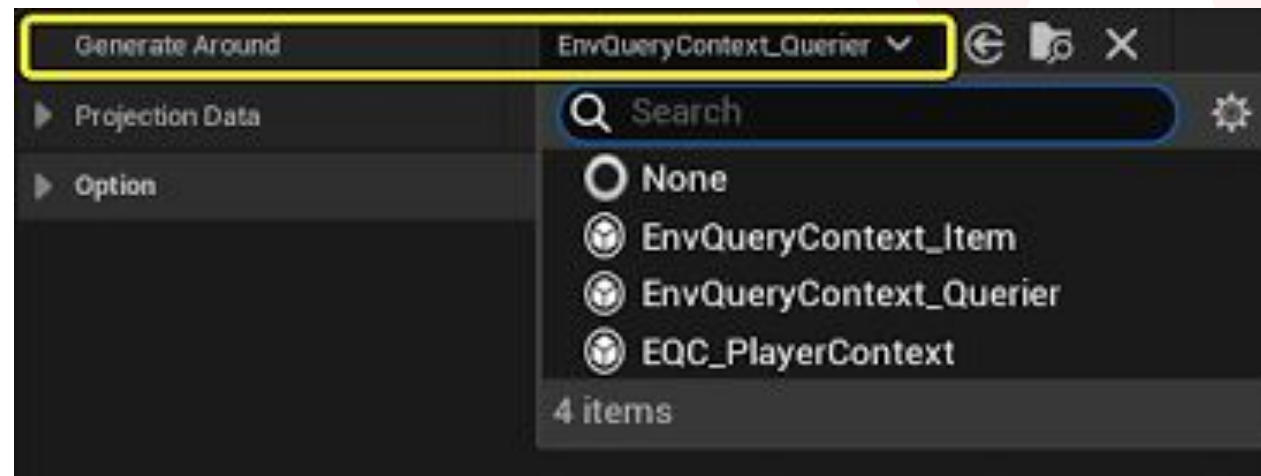
EQS - Context

Un **Context** fornisce un punto di riferimento per i vari test e generatori. Può variare dal *querier* – ovvero il Pawn attualmente posseduto dall'AI Controller che esegue il Behavior Tree – fino a scenari più complessi che coinvolgono tutti gli attori di un certo tipo.

Ad esempio, un *Generator* come *Points: Circle* può utilizzare un *Context* per ottenere più posizioni o attori.

Le classi di *Context* disponibili sono:

- **EnvQueryContext_Item**: Rappresenta una posizione (come un vettore) o un attore.





EQS - Context

- **EnvQueryContext_Querier**: Rappresenta il *querier*, ovvero il Pawn attualmente posseduto dall'AI Controller, che sta eseguendo il Behavior Tree.

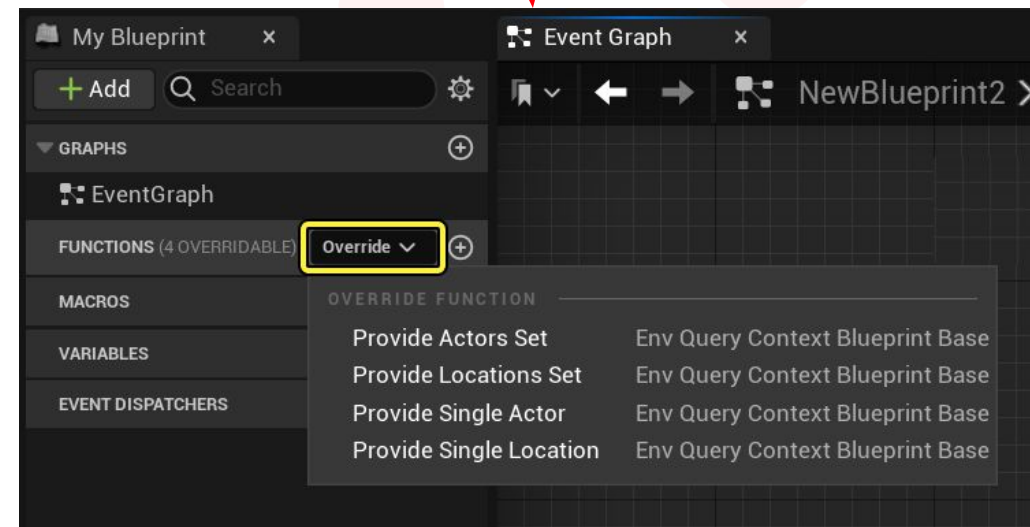
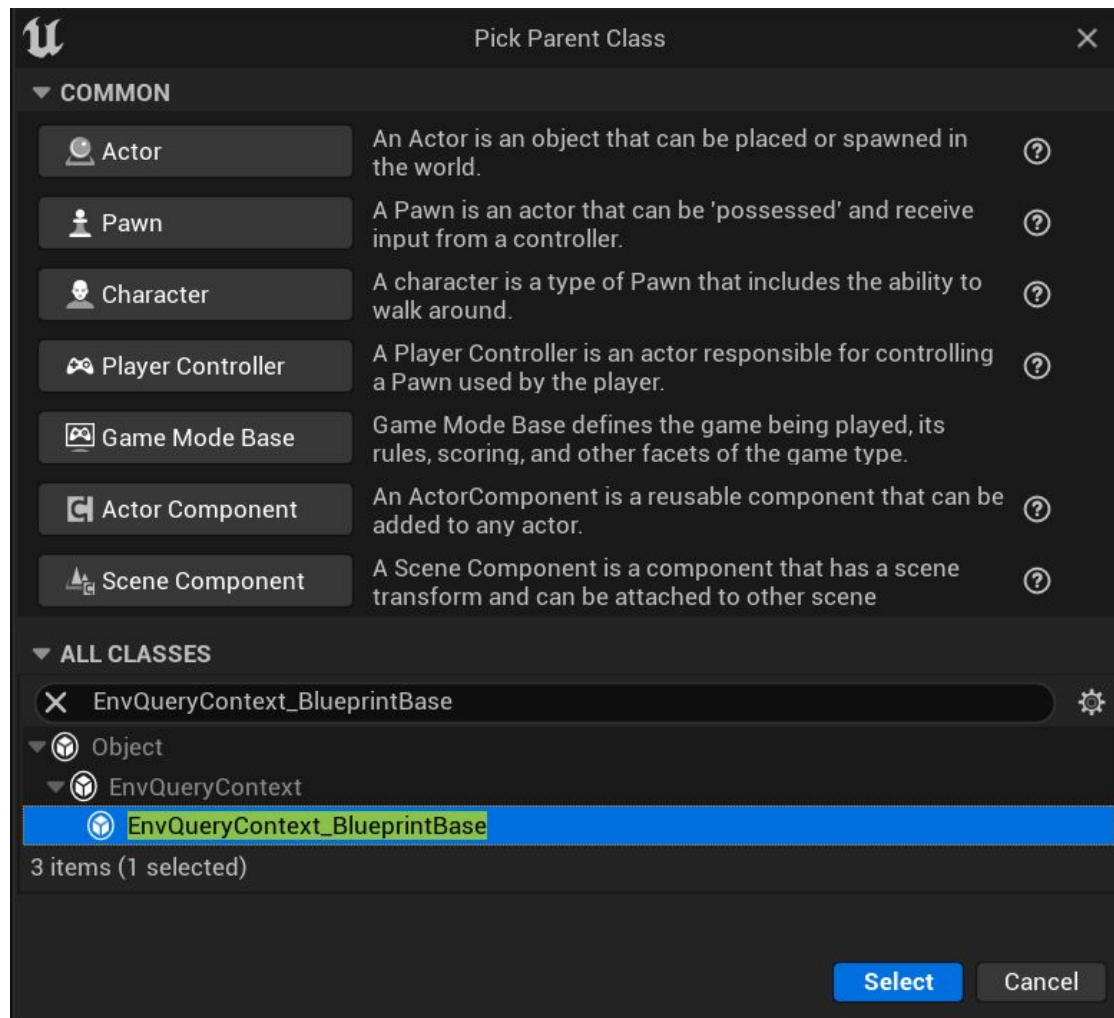
▼ Generator	
Searched Actor Class	Actor ▼ ⏪ 🔍 ⊕ ✕
▼ Generate Only Actors in Radius	☒
Data Binding	None ▼
▼ Search Radius	500.0
Data Binding	None ▼
Search Center	EnvQueryContext_Querier ▼ ⏪ 🔍 ✕

▼ Generator	
▶ Circle Radius	1000.0
▶ Space Between	50.0
▶ Number Of Points	8
Point on Circle Spacing Method	By Space Between ▼
▶ ☐ Arc Direction	between two contexts...
▶ Arc Angle	360.0
Circle Center	EnvQueryContext_Querier ▼ ⏪ 🔍 ✕
Ignore Any Context Actors when Gen...	☐
▶ Circle Center ZOffset	0.0



EQS - Context

Anche in questo caso, è possibile creare dei *Context* personalizzati estendendo la classe *EnvQueryContext* (se sviluppi in C++) o *EnvQueryContext_BlueprintBase* (se sviluppi con Blueprints).





EQS - Tests

Un **Test** determina i criteri utilizzati dalla query ambientale – ovvero la richiesta effettiva all'ambiente – per selezionare l'item ottimale generato, in base a un determinato contesto.

Ogni tipo di Test possiede proprietà uniche che permettono di definire come il test viene eseguito. Tuttavia, tutti i test condividono alcune proprietà comuni che determinano lo scopo del test e il modo in cui devono essere gestiti i risultati.

Questa proprietà determina le opzioni aggiuntive disponibili nel test e il suo scopo d'uso.

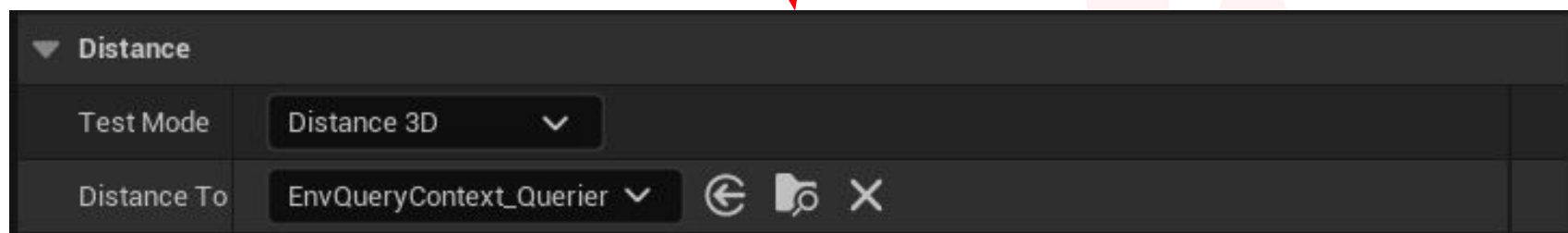
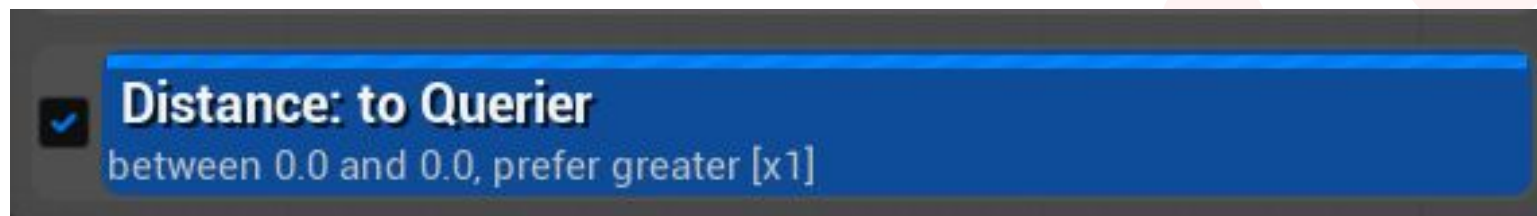
- **Filter Only:** Utilizzato per filtrare i risultati possibili. Gli items che non superano il test vengono rimossi.
- **Score Only:** Utilizzato per assegnare un punteggio ai risultati possibili. Gli items vengono restituiti con un valore di peso.
- **Filter and Score:** Utilizzato sia per filtrare che per assegnare un punteggio ai risultati.



EQS - Tests

Alcuni dei test disponibili out of the box includono:

- **Distance:** Restituisce la distanza tra la posizione dell'item e un'altra posizione.

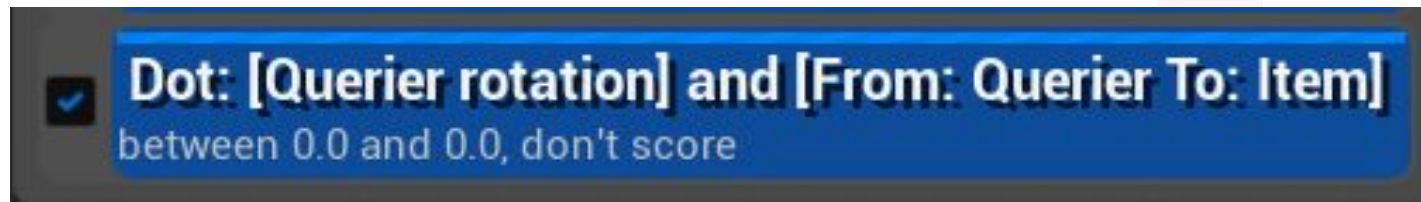




EQS - Tests

- **Dot:** calcola il prodotto scalare di due vettori. Questi possono essere le rotazioni di un contesto o un vettore da un punto a un altro.

Questo test è utile per determinare se un oggetto è rivolto verso un altro.

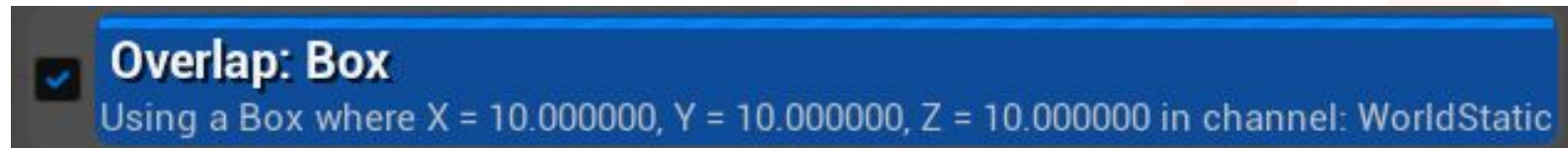


▼ Dot	
▼ Line A	context's rotation...
Mode	Rotation ▼
Rotation	EnvQueryContext_Querier ▼ ↶ ↷ ✕
▼ Line B	between two contexts...
Mode	Two Points ▼
Line From	EnvQueryContext_Querier ▼ ↶ ↷ ✕
Line To	EnvQueryContext_Item ▼ ↶ ↷ ✕
Test Mode	Dot (3D) ▼
Absolute Value	☐



EQS - Tests

- **Overlap: Box:** Verifica se un *item* si trova all'interno dei limiti definiti dal test stesso.

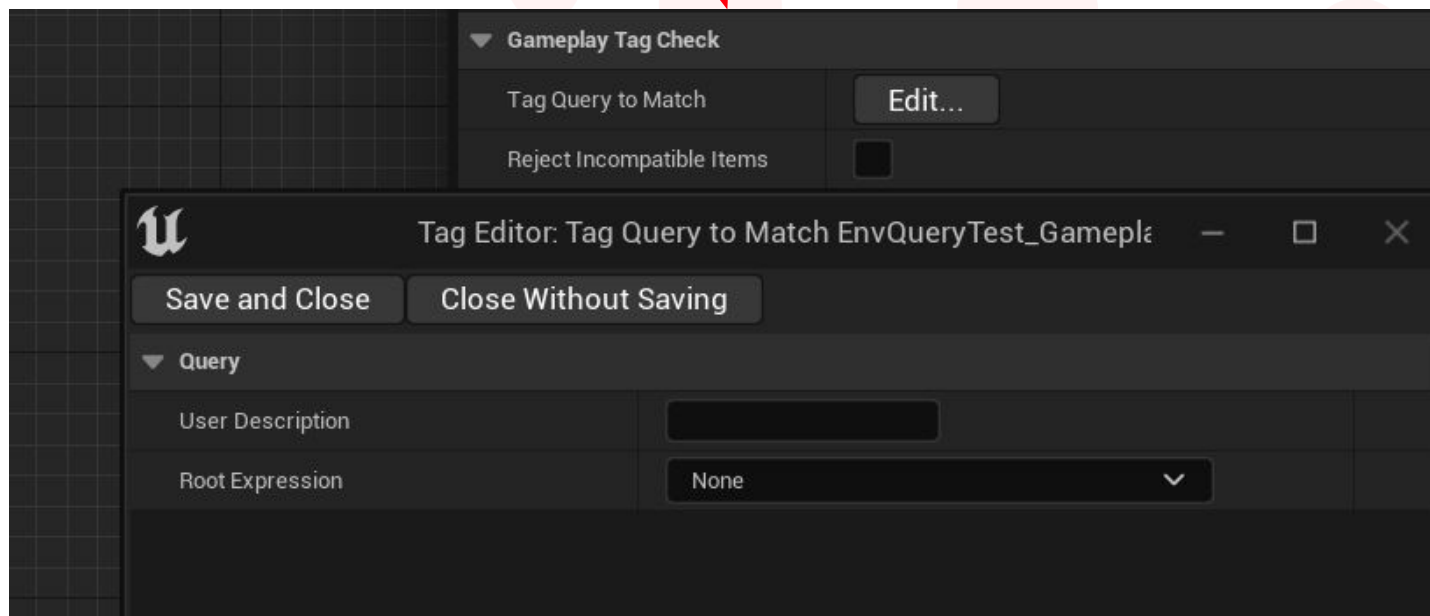


▼ Overlap			
▼ Overlap Data			
Extent X	10.0		
Extent Y	10.0		
Extent Z	10.0		
▶ Shape Offset	0.0	0.0	0.0
Overlap Channel	WorldStatic ▼		
Overlap Shape	Box ▼		
Only Blocking Hits	☒		
Overlap Complex	☐		
Skip Overlap Querier	☐		



EQS - Tests

- **Gameplay Tags:** permette di specificare un Tag da interrogare e confrontare all'interno del test.





EQS - Tests

- **PathExists: from Querier:** Controlla se esiste un percorso verso il contesto e restituisce informazioni utili, come la lunghezza del percorso.

☒ **PathExist: from Querier**
discard unreachable
require existing path, constant score [x1]

▼ Pathfinding

Test Mode	Path Exist ▼
Context	EnvQueryContext_Querier ▼ ⏪ 📁 ✕
▶ Path from Context	☒
Filter Class	None ▼ ⏪ 📁 + ✕
▼ Advanced	
▶ Skip Unreachable	☒



EQS - Tests

- **Project:** può essere utilizzato per proiettare gli items risultanti sulla NavMesh (specificando anche quale dataset della NavMesh utilizzare).

Questo test sposta gli items che potrebbero trovarsi dentro muri o aree bloccate, riportandoli sulla NavMesh. Tuttavia, ciò può causare un raggruppamento se una linea della griglia si trova appena oltre il bordo della NavMesh.

☒ **Project**
require projected, constant score [x1]

▼ Test

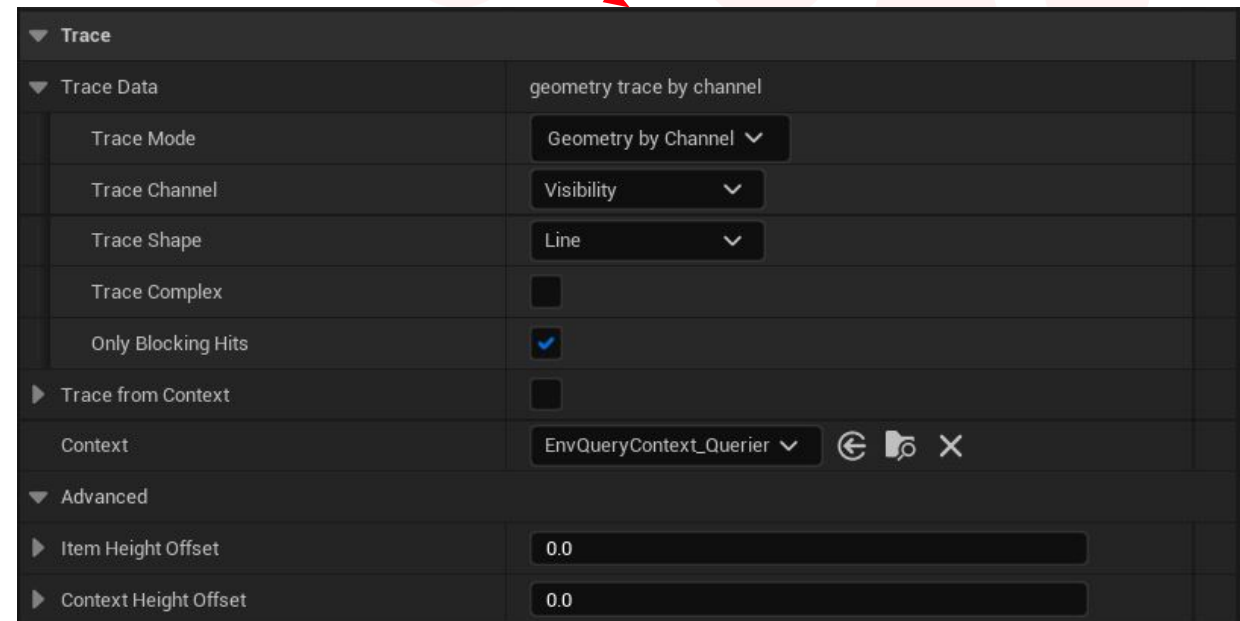
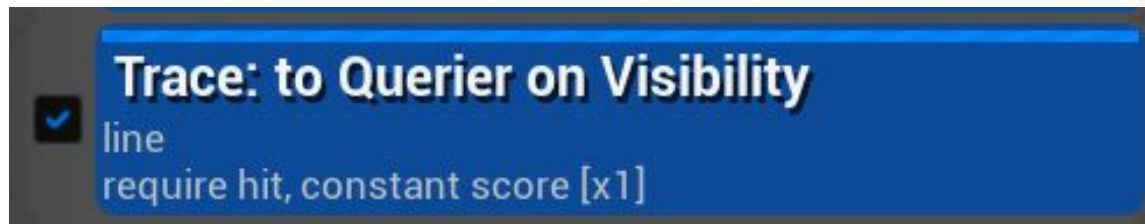
▼ Projection Data	navmesh trace
Trace Mode	Navigation ▼
Navigation Filter	None ▼
Extent X	0.0
Project Down	1024.0
Project Up	1024.0
Post Projection Vertical Offset	0.0
Test Comment	
Test Purpose	Filter and Score ▼



EQS - Tests

- **Trace:** esegue un trace verso (o da) un *Item* o un *Context* e restituisce se ha colpito qualcosa o meno. Puoi invertire il risultato utilizzando l'opzione Filter, con il parametro *Bool Match*.

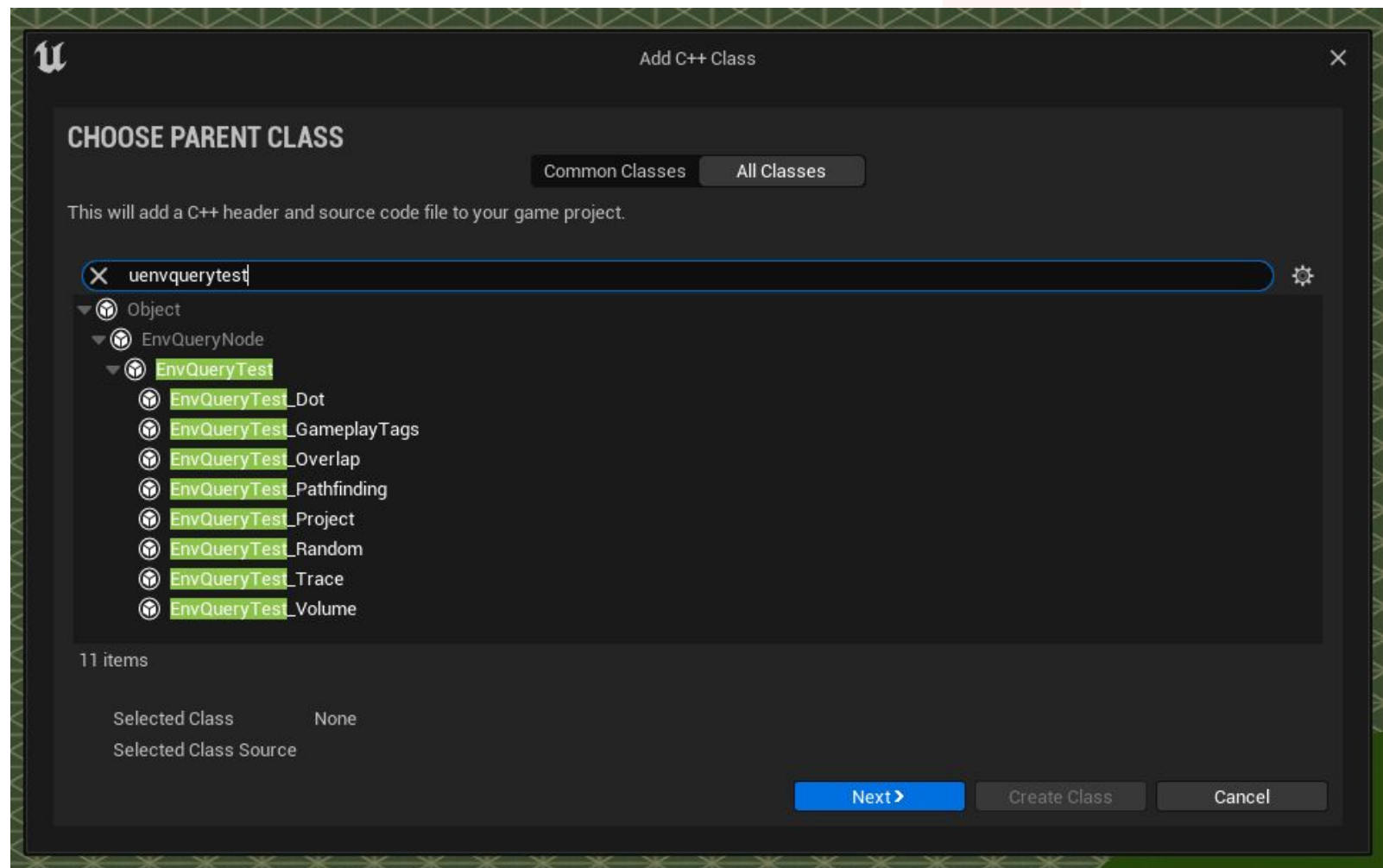
Un caso d'uso tipico per questo test è determinare se un nemico può (o non può) vedere un Player nel livello.





EQS - Tests

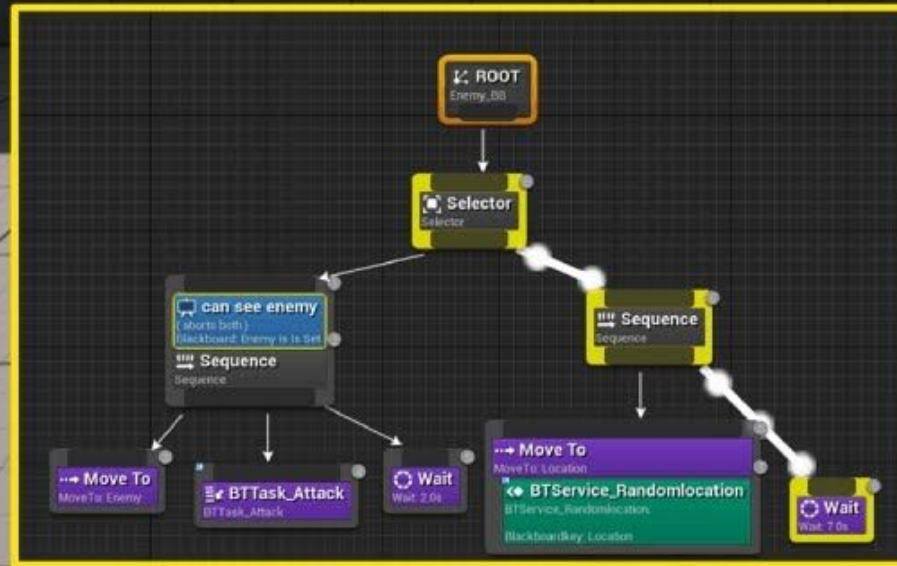
E, ovviamente, puoi anche implementare test personalizzati estendendo la classe *EnvQueryTest*, sia in C++ che in Blueprints.





A.I BEHAVIOR

TREE



PRESS START



Behaviour Tree - Esercizio

Riprendiamo il nostro NPC che dal suo punto di partenza deve arrivare al suo rifugio il prima possibile.

Questa volta utilizziamo i BT per far muovere la nostra AI, posizionando però vari waypoint in differenti terreni, così da dover creare un Task per trovare il prossimo punto più vicino.

Inoltre, nel muoversi, essa perderà Stamina e se questa arriva a 0, l'NPC dovrà fermarsi a recuperare il fiato.

In aggiunta, creiamo una Service che controlli lo stato della Stamina e modifichi la velocità del personaggio se questa è minore o maggiore al 30% (camminata o corsa).

Successivamente, per estendere ulteriormente l'esercizio, andiamo ad aggiungere degli actor in scena che fungeranno da punti di riposo (*RestArea*) dove la Stamina verrà ricaricata più velocemente.

Utilizzare un EQS per trovare il punto di riposo più vicino all'NPC nel momento in cui quest'ultimo è troppo stanco per correre.

